

Meta-Learning Orchestration of Learning Modalities on Resource-Constrained Robots: A Systematic Review

Yeshwanth Guru and Dev Kunwar Singh Chauhan

Abstract—A low-cost robot that must perceive, learn from demonstration, improve through reinforcement and recover from faults on its own hardware cannot run every learned model at every step. It needs an orchestrator: a component that decides which learning modality to invoke, how much computation to spend and how far to trust the result, and that improves these decisions from experience. This systematic review asks whether the literature supports such a meta-learned orchestrator on resource-constrained hardware. Following PRISMA 2020, 180 works were included from a curated list, structured searches and a targeted search closed on 4 October 2026. Each work was graded on a six-level evidence scale, coded for the platform its inference needs and for the confidence signal it exposes, and the 161 primary studies were appraised against seven criteria. The components exist separately: calibrated or epistemic confidence signals, conformal help-seeking, budget-aware routers, adaptive computation inside policies, meta-learned adaptation and on-device fine-tuning. Their combination does not. No included work arbitrates between more than two heterogeneous learning modalities with a learned, calibrated and compute-budgeted gate; of the 65 real-robot studies, only five run inference on a Raspberry-Pi-class processor and only two of these expose a confidence signal. Most systems need a workstation GPU (68 works) or a datacenter (34). The review formalizes orchestration as confidence-driven routing in a constrained Markov decision process and treats the orchestrator as a meta-learner. An illustrative simulation shows that miscalibration lowers routing success from 0.85 to 0.81 and that a calibration prior meta-learned across tasks recovers most of the loss within the first 25 steps on a new task (0.82 against 0.77 when learning from scratch). The review closes with an evaluation protocol for real robots and five open problems.

Index Terms—Adaptive computation, edge computing, embodied agents, meta-learning, mixture of experts, robot learning, systematic review, uncertainty quantification.

I. INTRODUCTION

A. Adaptive Embodied Agents on Resource-Constrained Hardware

A household or laboratory robot faces a stream of situations that no single learned policy handles well. It must recognize objects it has not seen, refine a skill that almost works, copy a behavior a person has just demonstrated, and keep functioning when a wheel slips or a joint degrades. Robot learning has developed a separate family of methods for each

of these needs. Uncertainty-aware perception [1], [2], deep reinforcement learning (RL) [3], [4], imitation learning [5], [6] and fault adaptation [7], [8] have each produced strong results. Two recent developments raise the question of how to combine them. Large language models (LLMs) can decompose tasks and call skills or tools [9]–[11], and generalist vision–language–action (VLA) models attempt to replace the separate modules with a single network [12]–[14].

Both routes are hard to realize on low-cost hardware. The LLM planners in the literature typically call cloud-hosted models, and the strongest VLAs have billions of parameters and need GPU inference (Section VI). Low-cost platforms such as the LeKiwi mobile manipulator from the open-source LeRobot ecosystem [15], [16] carry a single-board computer, two cameras and a small arm. An agent on such a platform cannot run every model all the time. It must decide which learning modality to use, when to spend computation, when to trust a module’s output, and how to improve those decisions from experience. This review calls the component that makes these decisions the *orchestrator*, and asks what the literature offers for building it.

B. Organizing Framework: Four Modalities, Their Confidence Signals and a Meta-Learned Gate

The review is organized around a design question: can an embodied agent on resource-constrained hardware adapt better if a small, local, trainable orchestrator learns to allocate computation across several learning modalities, using calibrated confidence signals that each modality emits? Four modality–signal pairs are considered: vision perception with epistemic uncertainty; RL with policy variance; imitation learning with behavioral fidelity; and fault adaptation with recovery-success rate. The orchestrator is framed as a *meta-learner*: it does not learn a task but learns, across a distribution of tasks, how to use the learners it has, and it adapts that knowledge to a new task or robot condition from a few outcomes. Fig. 1 shows the architecture used as the lens of the review. For each component, the review asks what the literature already provides, what is missing and what would have to be shown for the composition to work.

C. Scope

The review is restricted to a single robot. “On-device” covers computation on the robot and on a local computer

Y. Guru and D. K. S. Chauhan are with the Department of Mechanical Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Chennai 601103, India (e-mail: g_yeshwanth@ch.students.amrita.edu; c_devsingh@ch.amrita.edu). Corresponding author: Yeshwanth Guru. ORCID: Y. Guru 0009-0007-6353-4033; D. K. S. Chauhan 0000-0002-1466-4567.

Manuscript received [FILL: date].

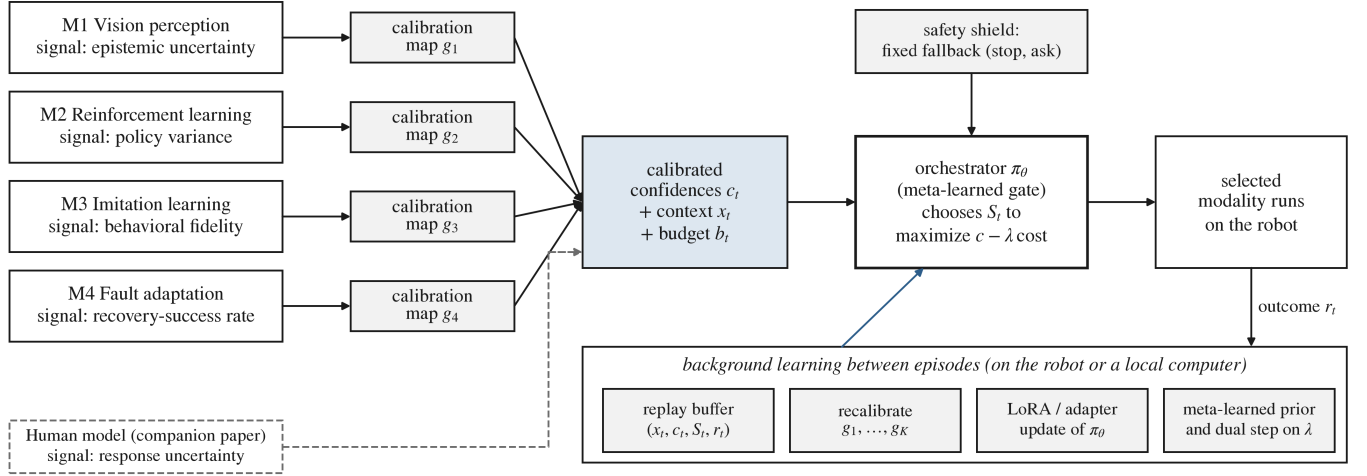


Fig. 1. Architecture used to organize the review. Each learning modality emits a confidence signal that a per-modality calibration map turns into an estimate of the probability of success. The orchestrator, a small meta-learned gate, chooses the modality to run within a compute budget, subject to a fixed safety fallback. Outcomes are stored and used between episodes to recalibrate the signals, update the gate’s adapter and refine the meta-learned prior. The dashed input is the human-model signal developed in the companion paper [17].

connected to it, but never a remote cloud service. This distinction matters in practice: the standard configuration of the LeKiwi mobile manipulator runs a Raspberry Pi 5 on the robot and streams observations over ZeroMQ to a laptop, where heavier policies run [15]. Multi-robot federated learning is discussed only as a source of fusion ideas (Section XII-C). Quantum and neuromorphic hardware are outside the scope; quantum technologies for robotics and quantum-like models of decision-making are reviewed in two companion surveys by the same authors [18], [19].

D. Research Questions

Five research questions structure the review:

- **RQ1.** Which learning modalities can a resource-constrained robot draw on, and which confidence signal can each modality emit? (Sections IV, VII–X)
- **RQ2.** How is the choice among models, skills or modalities made in current systems, and is that choice learned, uncertainty-aware, compute-budgeted and deployable on edge hardware? (Sections V, VI, XIV)
- **RQ3.** Which confidence signals are cheap and calibrated enough to be compared and fused on edge hardware? (Sections VII–X, XII)
- **RQ4.** Can meta-learning and on-device adaptation train such an orchestrator on the robot within its memory and compute? (Sections XI, XIII)
- **RQ5.** How far does the published evidence go toward a physical, resource-constrained robot, and what evaluation would show that orchestration outperforms its strongest single component? (Sections III, XV)

Section XVI answers each question.

E. Related Surveys and Contributions

Existing surveys cover the components of the problem separately: uncertainty in deep networks [20], meta-learning [21],

continual learning [22], multimodal learning [23], federated learning [24], [25], imitation learning [26], safe learning in robotics [27], [28], fault detection in robots [29], anomaly detection [30] and efficient VLA models [31]. None of them addresses how heterogeneous learners should be orchestrated on resource-constrained hardware, and none grades its corpus by how close the evidence comes to such hardware. The review makes six contributions.

- A modality-by-signal taxonomy that pairs each learning modality with a confidence signal it can emit cheaply (Table III).
- A systematic, reproducible review of 180 works, each graded on a six-level evidence scale, coded for the platform its inference needs and the confidence signal it exposes, and appraised against seven criteria; the data and coding rules are released (Section II).
- An evidence profile showing where the literature stops short of the target: real-robot studies rarely run on edge processors and rarely expose a confidence signal (Section III, Fig. 4).
- A gap analysis and a formal problem statement of orchestration as budget-constrained, confidence-driven routing, with the orchestrator cast as a meta-learner (Sections XI and XIV).
- An illustrative simulation of the effect of miscalibration on routing and of a meta-learned calibration prior on adaptation to a new task (Section XIV-B, Appendix B).
- An evaluation protocol for real-robot studies and five open problems (Sections XV and XVII).

F. Organization

Section II describes the review method and Section III the evidence profile of the corpus. Section IV introduces the four learning modalities. Sections V and VI review LLM orchestrators and VLA models. Sections VII–X review the

four modalities and their confidence signals. Sections XI–XIII cover meta-learning, uncertainty fusion and on-device adaptation, and Section XIV covers adaptive computation, mixture of experts and the formal problem statement. Section XV discusses low-cost platforms and evaluation, Section XVI answers the research questions, Section XVII lists open challenges and Section XVIII concludes.

II. REVIEW METHOD

The review follows the PRISMA 2020 statement [32] and the guidelines of Kitchenham and Charters for systematic reviews in computing [33]. It was registered retrospectively on the Open Science Framework **[FILL: OSF registration DOI]**; the protocol text, the search record and the retraction check are released with the data. Fig. 2 shows the flow of records.

A. Eligibility Criteria

A work was included if it (I1) proposed, evaluated or reviewed a method in one of twelve component areas of learned orchestration for embodied agents: LLM planners and orchestrators; VLA and robot foundation models; perception with epistemic uncertainty; RL and policy variance; imitation learning and behavioral fidelity; fault detection, adaptation and recovery; meta-learning; uncertainty fusion and calibration; on-device adaptation and continual learning; adaptive computation, mixture of experts and routing; low-cost platforms, benchmarks and evaluation; and safe learning; (I2) reported the method and its evaluation in enough detail to be graded, or was a review, benchmark or software release that defines a component; and (I3) was a journal article, conference paper or arXiv preprint in English published between 1991 and October 2026. A work was excluded if it (X1) concerned multi-robot or fleet learning without a single-robot component, (X2) concerned quantum or neuromorphic hardware, (X3) could not be located or verified on an arXiv or publisher page, (X4) was not a scholarly source (blog, product page, talk abstract or model card) or (X5) duplicated an included work, in which case the archival version was kept.

B. Information Sources and Search

The corpus was assembled in three stages. Stage 1 was a curated reading list of 108 works, assembled while the orchestration framework was designed. Every item was checked against its arXiv or publisher record, and 30 bibliographic details (year, venue, title or authors) were corrected. Stage 2, in September 2026, comprised structured searches of arXiv, the proceedings of CoRL, RSS, ICRA, IROS, NeurIPS, ICML and ICLR, and publisher sites in ten topic streams: uncertainty-aware LLM and VLA planning; efficient and edge VLAs; budget-aware model routing; RL fine-tuning of large policies; failure detection and recovery with foundation models; uncertainty fusion and calibration; continual and on-device learning; adaptive computation in robot policies; safe learning; and low-cost hardware and evaluation methodology. Backward citation chasing from the reviews in the corpus supplemented

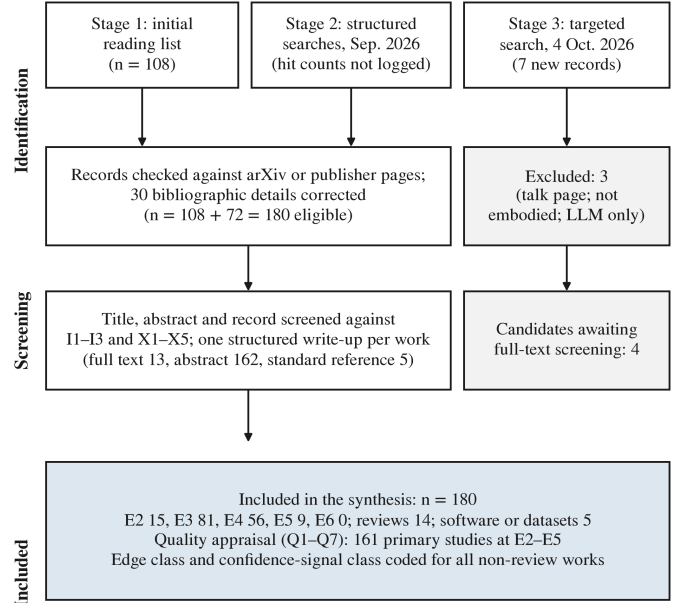


Fig. 2. Flow of records through the review (PRISMA 2020). Search hit counts were not logged for stage 2. The four stage-3 candidates await full-text screening and are not counted in the corpus.

the streams, and 72 works were added. The exact strings and hit counts of stage 2 were not logged; the concept blocks that reproduce their scope are given in the supplementary material (Section S2). Stage 3 was a logged targeted search on the title topic, run on 4 October 2026. It returned seven new records: three were excluded (a talk page, a non-embodied agent and an LLM-only study) and four were recorded as candidates for full-text screening and are not counted in the corpus. The corpus was closed on 4 October 2026. All 180 references were matched against the Retraction Watch Database (72,870 records, current to 22 September 2026), and none has been retracted.

C. Data Extraction and Coding

For every work, a structured write-up recorded the task, method, setup and data, key result, limitation, compute footprint relative to a Raspberry-Pi-class robot and role in the orchestration framework. Thirteen write-ups were made from the full text, 162 from the abstract and the publisher or arXiv page, and five from the standard description of well-known works; numbers not stated in the abstract are marked as such and were not inferred. Two attributes were then hand-coded for every work that is not a review. The *edge class* records the platform that the system’s inference needs as published: Raspberry-Pi-class or embedded CPU; embedded GPU (Jetson class); workstation GPU (a desktop, laptop or single consumer GPU); datacenter or cloud; or not stated. The *confidence-signal class* records the signal the work provides or uses: conformal or explicitly calibrated; epistemic uncertainty (ensembles, dropout, evidential or Bayesian posteriors); failure, anomaly or novelty score; policy variance or entropy; routing score or

predicted quality; or none. The extraction table, summaries and coding files are released (Data Availability).

D. Evidence Levels

Every work was graded on the scale of Table I. The levels follow the steps a component must pass before it can be relied on in an orchestrated robot: it is proposed (E1), specified as an algorithm or analysis (E2), evaluated in simulation or on a benchmark (E3), shown on a physical robot (E4), shown on several platforms or at scale (E5) and used with real users over time (E6). The scale is ordinal and was designed for this review. It grades how far the evidence goes toward a deployed robot, not the scientific quality of the work, and ambiguous cases received the lower level. A second rater re-graded a random sample [FILL: second rater; sample size; Cohen's kappa for evidence levels and edge class].

E. Quality Appraisal

The 161 primary studies at levels E2–E5 were appraised against seven criteria chosen for the orchestration question: Q1 method specification; Q2 comparison with a named baseline; Q3 physical validation; Q4 reporting of compute, latency or memory; Q5 quantitative evaluation; Q6 release of code or models; and Q7 a confidence signal that an orchestrator could read. Each criterion was coded as met (M), partly met (P), not met (N) or not assessable without the full text (?), by explicit rules applied to the extraction fields; the rules are listed in the codebook of the data package, and ambiguous cases received the lower code. Because most codes rest on abstracts, the appraisal is a first pass, to be confirmed against the full texts. Table II gives the results. Most studies specify their method (Q1 met in 104) and report a result with a number (Q5 met in 68), but only 36 report compute, latency or memory as a number (Q4), and only 36 expose a calibrated, epistemic, failure or policy-variance signal (Q7). Code release (Q6) could be confirmed for only six, because abstracts rarely state it.

F. Synthesis and Threats to Validity

Outcomes are too heterogeneous for meta-analysis, so the synthesis is narrative and organized by the framework of Fig. 1, supported by counts by evidence level, edge class and signal class. Several threats apply. The field moves quickly, and 25 of the included works were preprints at the time of the search. Most write-ups rest on abstracts, which can misstate the evidence level or omit compute figures; this biases Q4 and Q5 downward and is why the lower level or code was assigned in doubtful cases. Screening and extraction were done by one reviewer, with the evidence levels and coding of the core works checked by the second author. The stage-2 search strings were not logged, so that stage cannot be reproduced exactly; the concept blocks and the logged stage-3 search partly compensate. The edge class records what the published system used, not what a smaller variant might achieve, and compute figures were not measured on a common platform. Finally, the illustrative simulation uses synthetic data and makes no claim about robot performance.

III. EVIDENCE PROFILE OF THE CORPUS

Fig. 3 shows the evidence levels by topic. The corpus is dominated by simulation and benchmark studies (81 works at E3) and laboratory robot studies (56 at E4); nine works reach E5 and none reaches E6. Real-robot evidence is concentrated in VLA models, fault detection and recovery, and LLM planning. In contrast, on-device adaptation and adaptive computation, the two areas that an edge orchestrator depends on most, are evaluated almost entirely in simulation or on benchmarks (19 of 23 and 14 of 20 works at E3).

Fig. 4 cross-tabulates the evidence level with the edge class and the confidence-signal class. Two patterns stand out. First, the works that run on a Raspberry-Pi-class processor are mostly algorithms and simulation studies (41 of 46 at E2 or E3); of the 65 real-robot studies (E4 and E5), 36 need a workstation GPU, 16 a datacenter or cloud service and only five run on a Pi-class processor: hindsight experience replay [40], intelligent trial and error [7], meta-learned dynamics for online adaptation [41], rapid motor adaptation [8] and ThriftyDAGger [42]. Second, 106 of the 166 non-review works expose no confidence signal at all. Among the real-robot studies, 14 expose a calibrated, epistemic, failure or policy-variance signal, and most of these are failure detectors and help-seeking planners that run on GPUs [37], [43]–[46]. Only two works are at once on a real robot, on a Pi-class processor and equipped with such a signal: intelligent trial and error, whose Gaussian-process posterior guides recovery after damage [7], and ThriftyDAGger, whose novelty and risk estimates decide when to hand control to a human [42]. Each arbitrates between two options. No included work arbitrates between more than two heterogeneous learning modalities with a learned, calibrated and compute-budgeted gate on any hardware.

IV. LEARNING MODALITIES IN ROBOTICS

A. Overview of the Four Modalities

Perception. Perception maps camera images and other sensor readings to object identities, poses and scene structure. Foundation vision–language encoders such as CLIP provide open-vocabulary recognition [47] and underpin language-conditioned manipulation [48]. For an orchestrator, the key property of a perception module is whether it knows when it does not know. Bayesian approximations, ensembles and evidential outputs provide this (Section VII) [1], [49], [50], and uncertainty surveys map the design space [20].

Reinforcement learning. Deep RL learns control from reward, from value-based methods with experience replay [51] to policy-gradient [4] and maximum-entropy actor–critic methods [3], [52]. Real-robot RL has reached 96% grasp success on unseen objects, but only with large-scale data collection [39]. Model-based RL improves sample efficiency across more than 150 tasks [53]. RL is the modality that can improve beyond its data, but it is the most expensive to run on hardware.

Imitation learning. Imitation learning copies expert behavior [26]. Its classical difficulty is compounding error under distribution shift, which DAGger addresses with interactive data aggregation [54]. Modern imitation, using diffu-

TABLE I
EVIDENCE LEVELS USED TO CLASSIFY THE REVIEWED WORKS, WITH THE NUMBER OF WORKS AT EACH LEVEL (N = 180)

Level	Label	Definition	Example	n
E1	Conceptual	Position or proposal without an algorithm or experiments	–	0
E2	Algorithmic or analytic	Algorithm or theory with analytic or toy evaluation	MC dropout [1]; shielding [34]	15
E3	Simulation or benchmark	Evaluated in simulation or on standard datasets	LIBERO [35]; RouteLLM [36]	81
E4	Real robot (laboratory)	Demonstrated on physical robots in the laboratory	Sentinel [37]; MoE-LoCo [38]	56
E5	Real robot (extensive)	Extensive real-robot evaluation or multiple platforms	OpenVLA [13]; QT-Opt [39]	9
E6	Deployment	Sustained deployment with real users	None found	0
R	Review	Surveys and tutorials	Uncertainty survey [20]	14
S	Software or dataset	Libraries, datasets and benchmark suites	LeRobot [16]	5

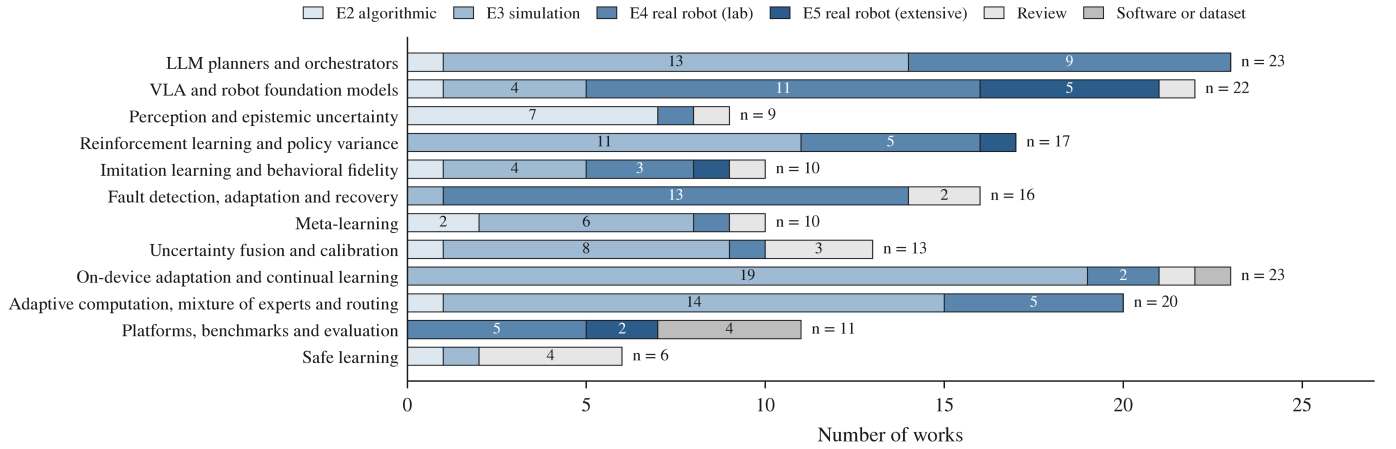


Fig. 3. Evidence levels of the 180 included works by topic. E2 algorithmic or analytic; E3 simulation or benchmark; E4 real robot (laboratory); E5 real robot (extensive or multi-platform). No work reaches E1 or E6.

(a) Inference platform as published					(b) Confidence signal provided or used				
Pi-class or embedded CPU	11	30	5	–	Conformal or calibrated	1	5	3	–
Embedded GPU	–	–	1	–	Epistemic uncertainty	5	7	2	–
Workstation GPU	1	27	29	7	Failure, anomaly or novelty score	1	2	9	–
Datacenter or cloud	1	17	14	2	Policy variance or entropy	–	1	–	–
Not stated	2	7	7	–	Routing score or predicted quality	2	15	7	–
	E2	E3	E4	E5	None	6	51	35	9
						E2	E3	E4	E5

Fig. 4. Evidence level of the 166 non-review works against (a) the platform their inference needs as published and (b) the confidence signal they provide or use. Hand-coded from the structured write-ups; coding rules in the data package.

sion policies [5], action-chunking transformers on low-cost hardware [6] and implicit energy-based models [55], learns dexterous skills from minutes of demonstrations. ACT reached 80–90% success on six tasks with ten minutes of demonstrations [6].

Fault adaptation and recovery. Robots that recover from damage use self-models [56], precomputed behavior repertoires searched by Bayesian optimization [7], meta-learned dynamics models that adapt online [41], or adaptation modules

trained in simulation that adjust within fractions of a second [8]. Detecting that something has gone wrong is itself a learning problem [29], [30], now addressed with foundation models (Section X).

B. Why Each Modality Alone Is Insufficient

Each modality has a characteristic failure. Perception is brittle out of distribution and often overconfident [57], [58]. RL is sample-hungry and unsafe to explore on hardware [27].

TABLE II
QUALITY APPRAISAL OF THE 161 PRIMARY STUDIES (E2–E5): NUMBER
OF STUDIES PER CODE

Criterion	M	P	N	?
Q1 Method specification	104	57	0	0
Q2 Baseline comparison	46	50	41	24
Q3 Physical validation	63	2	96	0
Q4 Compute reporting	36	19	106	0
Q5 Quantitative evaluation	68	30	0	63
Q6 Code or model release	6	0	0	155
Q7 Confidence signal	36	24	101	0

Imitation cannot exceed its demonstrations and degrades under distribution shift [54]. Fault adaptation methods are specialized to a platform or a class of damage [8]. Robustness baked in offline, for example by domain randomization [59], cannot anticipate every failure. No single modality covers the situations a long-lived robot will meet.

C. The Case for Composition

The strengths are complementary. Imitation gives fast competence, RL gives improvement [60], [61], perception grounds both, and fault adaptation keeps the robot operational. Several lines of work already combine two modalities. Residual RL refines imitation policies [60], [61], RL in the latent noise space steers diffusion policies [62], and interactive imitation switches control between robot and human under a budget [42]. What these combinations lack is a general, learned arbiter across more than two modalities that uses each modality’s own estimate of its reliability. That arbiter is the subject of Sections XI–XIV. Table III pairs each modality with the confidence signal it can emit and the cost of computing it.

V. LLMs AS PLANNERS AND ORCHESTRATORS

A. LLM-Driven Planning and Tool Use

LLMs were first shown to produce executable plans for embodied agents zero-shot, with a trade-off between executability and correctness [67]. SayCan grounded the choice among robot skills by multiplying LLM relevance with learned affordance values [9]. Inner Monologue closed the loop with textual feedback on success and scene state [68]. Code as Policies [69] and ProgPrompt [70] generated programs that call perception and control APIs. LM-Nav composed three frozen models for navigation [71], LLM-Planner re-planned on failure with little paired data [72], VoxPoser composed 3D value maps [73], and Language to Rewards let an LLM specify rewards for model-predictive control, solving 90% of tasks against 50% for a primitive-skill baseline [74]. Plan-Seq-Learn used an LLM for high-level plans and RL for low-level skills [75]. PaLM-E injected continuous observations into a 562-billion-parameter model [76], and Voyager built a growing skill library in Minecraft [77]. These systems establish that an LLM can decide which module to call. However, they rely on large cloud models and do not learn the calling policy from robot experience.

B. Agent Loops and Tool Calling

ReAct interleaves reasoning and acting, improving success by 34 and 10 percentage points on ALFWorld and WebShop over imitation and RL baselines [10]. Toolformer teaches a model, by self-supervision, when to call which tool [11]. This is the closest analog to learning when to invoke each modality, although it is offline and not embodied. Reflexion improves through verbal reflection stored in memory, without weight updates [78], and Tree of Thoughts searches over reasoning paths at much higher inference cost [79]. A second generation updates weights. FireAct fine-tuned a 7B model on agent trajectories [80]. AgentTuning instruction-tuned open models on agent data [81], Retroformer trained a small reflection model by policy gradient [82], and RL fine-tuning of vision–language models as decision agents outperformed GPT-4V on several tasks [83]. These results show that small orchestrators can be improved by fine-tuning, a prerequisite for learned orchestration.

C. Limits of a Frozen LLM Orchestrator

A frozen LLM orchestrator has three weaknesses in the edge setting. It does not know when it is wrong. KnowNo addressed this with conformal prediction, asking for help when the prediction set over plan options contains more than one element and providing statistical guarantees on task completion [45]. Introspective planning tightened these bounds [84], and the approach extends to multi-robot planning [85] and to deciding when to stop exploring [86]. A frozen LLM also cannot adapt its calling policy to the particular robot, whose modules succeed or fail in ways no pretraining corpus describes. Finally, its cost is fixed: it cannot decide to think less when the situation is easy. Recent work learns exactly this with an RL meta-policy that decides when to invoke expensive LLM reasoning. That meta-policy cut LLM inference time by more than 60% with a small loss in success [87].

D. Learned Versus Hard-Coded Orchestration

Table IV summarizes the orchestration literature along four axes. The axes are whether the orchestrator is learned, whether it uses calibrated uncertainty, whether it respects a compute budget, and whether it runs on edge hardware. Existing systems satisfy at most two. The closest are policy steering that decides whether to act, ask or request intervention [46]; fast–slow anomaly handling with small language models [88]; budget-aware reasoning [87]; and budgeted human–robot gating [42]. None of them arbitrates between heterogeneous learning modalities, and none learns the arbitration online on the robot. This is the gap targeted by the framework of this survey.

VI. VLA AND ROBOT FOUNDATION MODELS

A. Vision–Language–Action Models

VLA models map images and language directly to actions. RT-1 trained a transformer on large real-robot datasets [91], and RT-2 co-fine-tuned a vision–language model to transfer web knowledge to control, evaluated over 6,000 trials [12].

TABLE III
THE FOUR LEARNING MODALITIES AND THEIR CONFIDENCE SIGNALS

Modality	Confidence signal	Typical estimator	Extra compute	Calibration need
M1 Vision perception	Epistemic uncertainty	MC dropout [1], ensembles [49], evidential [50]	None (evidential) to $N \times$ (ensembles, MC)	Temperature scaling [57]
M2 Reinforcement learning	Policy variance	Entropy [3], critic or head ensembles [63], novelty [64]	None (entropy) to K heads	High: entropy is not a success probability
M3 Imitation	Behavioral fidelity	Action spread [37], density and OOD scores [43], [65]	Several samples or a small density model	Temperature [66]; conformal thresholds
M4 Fault adaptation	Recovery-success rate	Beta posterior over outcomes; anomaly scores	Negligible	Low (counts outcomes) but slow to accumulate

TABLE IV
ORCHESTRATION APPROACHES AGAINST FOUR REQUIREMENTS OF LEARNED EDGE ORCHESTRATION

Approach	Learned policy?	Calibrated uncertainty?	Compute budget?	Edge-deployable?
SayCan, Inner Monologue [9], [68]	No (frozen LLM)	No	No	No (cloud LLM)
Code as Policies, ProgPrompt [69], [70]	No	No	No	Generated code yes; LLM no
ReAct, Reflexion [10], [78]	In-context only	No	No	Depends on LLM
Toolformer [11]	Yes (offline)	No	No	Mid-size model
FireAct, Retroformer [80], [82]	Yes (fine-tuned)	No	No	Possibly (small models)
KnowNo, introspective planning [45], [84]	No	Yes (conformal)	No	Calibration yes; LLM no
Act, ask, or learn [46]	Partly (residual learning)	Yes (conformal)	No	No (large VLM)
Fast/slow anomaly handling [88]	Classifier trained	Partly	Implicitly (fast path)	Fast path plausibly
RouteLLM, Hybrid LLM, AutoMix [36], [89], [90]	Yes	Partly (predicted quality)	Yes	Router yes
RARRL [87]	Yes (RL)	No	Yes	Not stated
ThriftyDagger [42]	Thresholds	Novelty and risk	Yes (human budget)	Yes
Target (unoccupied)	Yes, online on device	Yes, across four modalities	Yes	Yes

OpenVLA, a 7B open model, outperformed the 55B RT-2-X by 16.5 percentage points in absolute success and can be fine-tuned with LoRA on consumer GPUs [13]. Octo [92], $\pi 0$ [14], RoboCat [93], Gato [94], VIMA [95], PerAct [96] and RoboFlamingo [97] extend the generalist program, and cross-embodiment datasets support it [98].

B. Limitations of Generalist Policies on Edge Hardware

Generalist policies are monolithic. They offer no handle for allocating computation across sub-skills, no native uncertainty estimate, and no mechanism for on-device improvement beyond offline fine-tuning. RL fine-tuning methods address the last point, for example raising OpenVLA’s real-world success from 40% to 70% in 40 minutes [99], reaching 96.3% average success after 45–90 minutes of online training [100], and scaling RL for VLAs [101], [102]. All of these need GPUs.

C. Memory, Latency, and Compute

Fig. 5 compares model sizes with the memory of a 4 GB Raspberry Pi 5. Storing the weights of a 2-billion-parameter model in 16-bit precision alone would fill that memory. OpenVLA (7B) and larger models therefore require an off-board GPU. The LeRobot paper reports that ACT needs about 5 ms and 211 MB per inference on an RTX 4090, against about 209 ms and 13.3 GB for $\pi 0$ [15]. Efficiency work narrows the gap: SmolVLA performs comparably to VLAs about ten times larger and runs on consumer GPUs or CPUs [103];

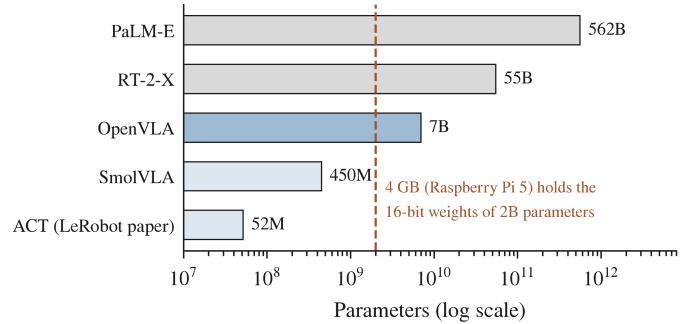


Fig. 5. Parameter counts of representative planners and policies (log scale) compared with a 4 GB Raspberry Pi 5, whose memory holds the 16-bit weights of at most about 2 billion parameters before any activations or operating system. Sizes from the cited papers; ACT’s size as reported in the LeRobot paper.

TinyVLA [104] and EdgeVLA [105] target fast inference; OpenVLA-OFT achieved 26 times higher throughput [106]; FAST compresses action tokens [107]; 1-bit and post-training quantization cut memory by up to 11 times [108], [109]; and state-space backbones reduce latency [110]. A survey maps efficient VLA design [31]. Even so, only the smallest policies are plausible on the Pi itself.

D. Modular Policies With a Small Local Orchestrator

The evidence favors a modular design: small specialized policies (ACT-scale imitation, compact RL policies,

lightweight perception) plus a small orchestrator. Distilling planning ability into a 3B on-device model recovered over 93% of GPT-4o's performance [111], and small routers can decide when larger models are needed (Section XIV). The trade-off is that the robot loses the broad generalization of large VLAs. Whether orchestration recovers enough adaptivity to compensate is an open empirical question.

VII. VISION PERCEPTION WITH EPISTEMIC UNCERTAINTY

A. Perception Pipeline

The perception modality turns camera images into task-relevant state such as object classes, masks and poses. On a low-cost robot with two RGB cameras and no depth sensor, 2D detection and segmentation with language-conditioned encoders [47], [48] are the realistic options. Voxel-based methods such as PerAct assume depth input [96]. Domain randomization yields detectors robust enough for 1.5 cm localization in clutter [59].

B. Epistemic Uncertainty Quantification

Epistemic uncertainty reflects what the model does not know and should shrink with more data, whereas aleatoric uncertainty reflects inherent noise [2]. Only epistemic uncertainty tells an orchestrator that another modality, or more data, could help.

Monte Carlo dropout. Keeping dropout active at test time and averaging several stochastic passes approximates Bayesian inference [1]. It needs no retraining, but its cost grows linearly with the number of passes, and the quality of its uncertainty is debated.

Ensembles. Deep ensembles give well-calibrated uncertainty, as good as or better than approximate Bayesian networks, and higher uncertainty on out-of-distribution inputs [49]. Their cost scales with the number of members, which is the central edge trade-off. Shared-backbone ensembles (multiple heads) reduce the cost, as bootstrapped DQN does in RL [63].

Evidential deep learning. Evidential networks output the parameters of a distribution over predictions in a single pass: a Dirichlet for classification [50] or a normal-inverse-gamma for regression [112]. This makes them the cheapest source of epistemic confidence and therefore attractive for a Raspberry Pi. Their calibration can degrade out of distribution. The simplest out-of-distribution signal, the maximum softmax probability, costs nothing extra but is often overconfident [58].

C. Calibration

Modern networks are systematically overconfident, and a single temperature parameter fitted on held-out data corrects much of this [57]. For VLAs, confidence predicts task success poorly without correction; prompt ensembles and Platt scaling improve it [113]. Calibration is a prerequisite for orchestration: a gate that compares confidences across modalities is only as good as their calibration (Section XII-B).

D. Cost Versus Uncertainty Quality on Edge Hardware

Table V compares the options. Temperature scaling and evidential outputs are essentially free, MC dropout and ensembles multiply inference, and conformal methods add a cheap calibration step on top of any score [45]. For a Pi-class robot, a practical design is an evidential or temperature-scaled detector, with ensembles reserved for offline calibration.

VIII. REINFORCEMENT LEARNING WITH POLICY VARIANCE

A. RL Backbones

Policy-gradient methods. Proximal policy optimization (PPO) is the default on-policy algorithm, balancing sample complexity, simplicity and wall-clock time [4]. Its small policy and value networks run comfortably on a Raspberry Pi, but training is usually done in simulation on a workstation.

Value-based and off-policy methods. DQN introduced experience replay for deep RL [51]. Off-policy actor-critic methods such as SAC [3] and TD3 [52] are more sample-efficient for continuous control. Hindsight experience replay makes sparse-reward robot tasks learnable by relabeling goals, and its simulated policies transferred to a physical robot [40]. Offline RL learns from fixed buffers without new interaction; conservative Q-learning often achieved two to five times higher final return than earlier offline methods [114]. This is relevant to learning from on-device logs. Model-based RL with world models improves sample efficiency [53], but its memory cost is high for edge training.

B. Policy and Value Variance as Confidence Signals

An RL policy's confidence can be read in two ways: from the spread of its action distribution, or from disagreement among value estimates.

Policy entropy. Maximum-entropy RL makes the policy stochastic by design, so its entropy is available at no extra cost [3]. Entropy, however, mixes aleatoric choice (several good actions) with epistemic ignorance (unknown state), and it is not calibrated to success probability. Calibration against observed outcomes is therefore needed before entropy is compared with other modalities' signals (Section XII-B).

Ensembles and twin critics. Bootstrapped DQN uses multiple heads on a shared network to approximate a posterior and drive deep exploration, improving learning times across most Atari games [63]. Head disagreement is a direct epistemic signal. TD3's twin critics reduce overestimation but do not quantify uncertainty [52]. Random network distillation provides a cheap novelty score for states the agent has not visited [64], which could also trigger the fault modality. For the orchestrator, a small ensemble of critics or a novelty score is a practical policy-variance signal.

C. Sample Efficiency on Edge Hardware

Training RL on the robot is limited by sample efficiency and safety. Three strategies reduce the burden. The first is refining an imitation policy with a small residual RL policy [60], [61]. The second is running RL in the latent noise space of a

TABLE V
UNCERTAINTY ESTIMATORS: COST AND SUITABILITY FOR A RASPBERRY-PI-CLASS ROBOT

Method	Inference cost	Strength	Weakness
Max softmax [58]	1 pass	Free	Overconfident
Temperature scaling [57]	1 pass + scalar	Fixes global miscalibration	Needs held-out data; not per-input epistemic
Evidential [50], [112]	1 pass	Epistemic signal in one pass	Can be miscalibrated out of distribution
MC dropout [1]	T passes	No retraining	Latency; quality debated
Deep ensembles [49]	M models	Strong, well calibrated	M× memory and compute
Conformal prediction [45]	Negligible over base score	Distribution-free coverage	Assumes exchangeability

diffusion policy without changing its weights, which achieved sample-efficient real-world improvement [62]. The third is fine-tuning large policies with offline-to-online RL [99]–[102], which needs a GPU. On a Pi-class robot, only the first two are realistic on board; heavier updates must run asynchronously on a local computer (Section XIII-C).

IX. IMITATION LEARNING WITH BEHAVIORAL FIDELITY

A. Learning From Demonstration

Behavioral cloning. Behavioral cloning (BC) regresses actions on observations. Its compounding-error problem motivated DAGger, which provides no-regret guarantees by aggregating expert labels on the learner’s own state distribution [54]. Modern BC is expressive. Diffusion policies model multimodal action distributions and improved on prior methods by 46.9% on average [5]. ACT predicts action chunks on low-cost bimanual hardware [6]. Implicit BC handles discontinuous actions with energy-based models [55]. Language-conditioned BC generalized to 24 unseen tasks with 44% average success [115], and learning from unstructured play covered about four times more interaction space than demonstrations [116]. Best practice for learning from offline human demonstrations is documented in robomimic [117].

Inverse reinforcement learning. Adversarial imitation matches the expert’s state–action occupancy rather than its actions [118]. It is more robust to distribution shift in principle, but its adversarial training is unstable and sample-hungry on real robots.

Multimodal imitation. Action multimodality, where several distinct actions are correct, is handled by diffusion [5] and energy-based models [55]. Multimodal inputs such as language and vision are handled by VLAs and multimodal prompts [95].

B. Behavioral Fidelity as Confidence

Behavioral fidelity is the degree to which the policy is operating within the distribution of its demonstrations, where its actions are reliable.

Action-distribution spread. For generative policies, the spread of sampled actions is a natural signal. Runtime monitors use the temporal consistency of sampled action chunks [37] and action-chunk entropy [65] to detect erratic failures. Per-task temperature scaling of language-conditioned imitation policies improved success without retraining, for example from 79.6% to 97.1% on block stacking with distractors [66]. A caveat is that the spread of a diffusion policy

reflects both genuine multimodality and uncertainty, so it must be calibrated against outcomes.

Density and reconstruction scores. A second signal is how well the current observation can be reconstructed or density-estimated from demonstration data. FAIL-Detect learns such scores from successful demonstrations only and sets thresholds by conformal prediction; its flow-based density signal was the most consistently effective [43]. FIPER combines observation-space out-of-distribution scores with action entropy [65]. These are practical fidelity signals because they need no failure data.

C. Fast Adaptation From Few Demonstrations

One-shot imitation meta-learns to imitate from a single demonstration [119]. ACT learns from about ten minutes of demonstrations [6], and Mobile ALOHA’s co-training raised success by up to 90% with 50 demonstrations per task [120]. Meta-learned adapters improved forward transfer over LoRA and raised real-robot success from 60% to 84% with 20 demonstrations [121]. Few-shot imitation is therefore the modality through which the orchestrator can acquire new behaviors fastest.

X. FAULT ADAPTATION AND RECOVERY

A. Fault Detection

Classical fault detection and diagnosis for robots is reviewed by Khalastchi and Kalech [29], and deep anomaly detection by Pang et al. [30]. Foundation models have transformed the area. REFLECT summarizes multisensory experience for an LLM to explain failures and plan corrections [122]. AHA fine-tunes a VLM to detect and explain manipulation failures, beating GPT-4o by 10.3 percentage points [123]. RoboFAC improved failure-analysis accuracy by 34.1 points over GPT-4o [124]. SAFE reads a VLA’s internal features to predict failure across tasks [44]. Sentinel combines a cheap consistency check with a VLM progress check and detected 18% more failures than either alone [37]. Most relevant to edge deployment, Sinha et al. use a fast embedding-based anomaly classifier with small language models and fall back to slower generative reasoning only when needed [88]. This fast–slow pattern mirrors budgeted orchestration.

B. Online Adaptation to Damage or Malfunction

Resilient robots and meta-adaptation. Continuous self-modeling let a robot infer its damaged morphology and

synthesize new gaits [56]. Intelligent trial-and-error searches a precomputed behavior map with Bayesian optimization to find compensatory behaviors within a few trials [7]. Meta-RL for dynamic environments adapts a learned dynamics model within a few steps to broken legs or new terrain [41]. This is the closest prior art to combining fault adaptation with meta-learning, but it uses a single learner. RMA adapts legged locomotion in fractions of a second with a small onboard network [8], showing that real-time adaptation modules fit on embedded hardware.

Recovery strategies and replanning. Recovery can mean re-invoking a different modality, asking a human, or replanning. Language corrections from a user improved a hierarchical policy without extra teleoperation [125], and human-in-the-loop deployment learning reweights intervention data for continual improvement [126]. For the orchestrator, recovery is an action choice, and switching modality is a recovery strategy in its own right.

C. Recovery-Success Rate as Confidence

The fault modality's confidence is its empirical recovery-success rate for the current fault class, which can be maintained as a Beta posterior over successes and failures. Unlike the other signals, it is directly calibrated because it counts outcomes, but it is only available after enough attempts. Conformal thresholds give failure detectors false-alarm guarantees from successful data alone [43], [65].

XI. META-LEARNING: TRAINING THE ORCHESTRATOR TO LEARN

Meta-learning is the part of the framework that the title refers to. The orchestrator does not learn a task; it learns, across many tasks, how to use the learners it has, and it must adapt that knowledge quickly when the robot, the environment or a fault changes. This section reviews the meta-learning methods in the corpus (ten works, mostly at E3), states the orchestrator's meta-learning problem and asks which method fits the memory and compute of a Pi-class robot.

A. Foundations

MAML learns an initialization that adapts in a few gradient steps [127], Reptile does so with first-order updates that need no more memory than ordinary training [128], and prototypical networks adapt without gradients by comparing embeddings with class prototypes [129]. Learned optimizers meta-learn the update rule itself [130]. A survey organizes the field into what is meta-learned (initialization, optimizer, metric or model) and how [21].

B. Meta-Reinforcement Learning

RL² trains a recurrent policy whose hidden state implements a learning algorithm, so adaptation needs only forward passes [131]. PEARL infers a probabilistic task context and was 20–100 times more sample-efficient than earlier meta-RL [132], and VariBAD maintains a belief over tasks and acts under it [133]. Meta-learned dynamics models let a legged

robot adapt online to a broken leg or new terrain within a few steps [41]; this is the only real-robot meta-learning study in the corpus that runs at Pi-class cost, and it adapts a single learner. Meta-World showed that algorithms struggle even with ten training tasks [134], a warning for orchestrators trained on narrow task distributions.

C. The Orchestrator as a Meta-Learner

Let a task τ be a combination of robot, environment and fault condition, drawn from a distribution $p(\tau)$. In task τ , the orchestrator with parameters θ routes among the modalities and collects outcomes; let D_τ^{tr} be its first few routing decisions and outcomes, and $U_\tau(\theta, D_\tau^{\text{tr}})$ its parameters after adapting to them. Meta-training seeks

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{\tau \sim p(\tau)} \left[\mathcal{L}_\tau(U_\tau(\theta, D_\tau^{\text{tr}})) \right], \quad (1)$$

where $\mathcal{L}_\tau$ is the negative cost-penalized return of Section XIV-F on the rest of the task. The inner update U_τ is what distinguishes the methods: a few gradient steps on the gate or its adapter (MAML, Reptile), a recurrent state update (RL²), a posterior update over a latent task variable (PEARL, VariBAD) or a running estimate shrunk toward a meta-learned prior (Section XI-E).

The orchestrator has two meta-learning roles. In *routing*, given fused confidences and context, it selects the modality to call. Belief-conditioned action selection [133] and context inference [132] provide the template: the unknown reliability of each modality in the current situation is a latent task variable, and routing proceeds under its posterior. In *fast adaptation*, when a new task or robot condition appears, the orchestrator must adapt its routing within a few episodes. Meta-learned adapters for robot policies raised real-robot success from 60% to 84% with 20 demonstrations [121], and one-shot imitation shows the analogous result for policies [119]. No included work meta-learns a gate over heterogeneous learning modalities.

D. Which Meta-Learner Fits an Edge Orchestrator

Table VI compares the methods by what adaptation costs on the robot. The orchestrator itself can be small: its input is a context embedding, K confidences and a budget, and its output is one of K modalities, so a network of a few thousand parameters, or a LoRA adapter on a small language model, suffices. Meta-training can then run off-board in simulation, and only the inner update must run on the robot. On that criterion, methods whose inner update needs no gradients (RL², posterior inference, prototypes, prior shrinkage) are the cheapest, and Reptile is the cheapest gradient-based option; MAML's second-order meta-gradient matters only for meta-training.

E. Meta-Learned Calibration

Calibration is the part of orchestration most exposed to change: a new robot, a worn gripper or a different room shifts how far each modality's confidence can be trusted (Section XII-B). Learning the shift from scratch on each

TABLE VI
META-LEARNING METHODS JUDGED BY THE COST OF ADAPTING AN ORCHESTRATOR ON A PI-CLASS ROBOT

Method	Inner update on the robot	On-robot cost of adaptation	Fit for an orchestrator
MAML [127]	Few gradient steps from a learned initialization	Ordinary backpropagation through the gate	Good for a small gate; second-order meta-training stays off-board
Reptile, FOMAML [128]	Few gradient steps	Ordinary backpropagation; no second-order terms	Cheapest gradient-based option; suits LoRA adapters
Prototypical networks [129]	New prototypes from a few examples	One embedding pass; no gradients	Routing as nearest prototype over situation embeddings
RL ² [131]	Recurrent state update	Forward pass only	No on-robot gradients; bounded by the training distribution
PEARL, VariBAD [132], [133]	Posterior over a latent task variable	Encoder pass	Treats modality reliabilities as latent; gives uncertainty over the routing itself
Learned optimizers [130]	Learned update rule	Small recurrent network per parameter	Memory overhead; feasible only for small adapters
Meta-learned adapters [121]	Adapter fine-tuning from a meta-learned start	Adapter-sized backpropagation	Closest robot evidence (E4), but for a single policy
Meta-learned dynamics [41]	Online update of a model prior	Small network; Pi-class	Only Pi-class real-robot example; single learner

task costs routing errors while the estimates settle. Meta-learning offers a cheaper route. The systematic part of each modality’s miscalibration, such as the overconfidence of policy entropy or of imitation action spread, can be estimated across training tasks and used as a prior, so that only the task-specific remainder has to be learned on the robot. The second part of the illustrative simulation tests this idea (Fig. 6(b), Appendix B). On each new task, the offsets of the four modalities’ confidences are drawn around systematic values (0, 0.30, 0.25 and 0.05) with a task-to-task spread of 0.10. A gate that learns the offsets from scratch reached 0.77 success in its first 25 steps and 0.82 after 100 steps. A gate whose offsets started from a prior meta-learned on 50 training tasks reached 0.82 from the first steps, and a gate on raw confidences stayed at 0.79. The remaining gap to the oracle (0.85) is mostly the cost of the 5% exploration that keeps rarely chosen modalities calibrated (Section XIV-D). The inner update is a running mean, so the on-robot cost is negligible.

F. Training the Orchestrator on Task Distributions

Meta-training requires a distribution of tasks in which different modalities are best. Benchmarks such as Meta-World [134], LIBERO [35], CALVIN [135] and RL-Bench [136] provide simulated task families, but none varies which learning modality is appropriate or injects faults. A practical approach is to build the task distribution from simulation with injected faults, perception noise and miscalibrated confidences, meta-train the gate there, and adapt it on the robot with one of the inexpensive inner updates of Table VI. Whether a distribution built this way covers the shifts met by a real robot is untested.

XII. UNCERTAINTY FUSION AND DECENTRALIZED CONFIDENCE SIGNALS

A. Heterogeneous Confidence Signals

The four signals differ in scale, semantics and cost (Table III).

Vision epistemic uncertainty. Obtained by MC dropout, ensembles or evidential outputs, then temperature-scaled (Section VII).

RL policy variance. Obtained from policy entropy or critic-ensemble disagreement (Section VIII-B). It is unbounded and must be mapped to a probability of success.

Imitation behavioral fidelity. Obtained from action spread, density or reconstruction scores (Section IX-B). Uncertainty for flow-based VLAs can be estimated from disagreement between velocity fields, with better calibration and at least 22% fewer samples than baselines [137].

Fault recovery-success rate. Obtained as an empirical success rate (Section X-C); it is calibrated by construction but slow to accumulate.

B. Making Confidences Comparable

To be compared, all signals must be expressed as calibrated estimates of the same quantity, the probability that invoking this modality now will succeed. Temperature scaling [57], Platt scaling [113] and conformal prediction [43], [45], [138] provide the tools, but none addresses cross-modality calibration, where signals of different origin must be placed on one scale. No included work calibrates confidences from heterogeneous learners against a shared success variable on a robot; this is an open challenge (Section XVII-A). The simulation of Section XIV-B illustrates the stakes: overconfident RL and imitation signals reduced routing success from 0.85 to 0.81, and a per-modality linear recalibration fitted on a separate set restored 0.85.

C. Federated Learning and Decentralized Confidence Fusion

Federated learning versus confidence fusion. In classic federated learning, clients train locally and a server averages weights. FedAvg reduced communication rounds by 10–100 times over synchronous SGD [139], and FedProx handles heterogeneous clients [140]. Surveys cover the field [24] and its extension to RL [25]. In the orchestration setting, “federated” is used differently: modalities stay separate and share only confidence signals, not weights or data, with the

orchestrator. To avoid confusion with FL, the term “decentralized confidence fusion” is used here: each modality computes and calibrates its own signal locally, and only the scalar signals are aggregated.

Fusion methods. Three families are available. Uncertainty-weighted averaging learns weights from each head’s uncertainty, as in multi-task loss weighting [141]. Evidential fusion combines per-source Dirichlet evidence with Dempster–Shafer rules, giving calibrated multi-view uncertainty [142], [143], and dynamic fusion weights modalities by their estimated informativeness per sample [144]. Learned gating and attention treat fusion as routing (Section XIV). Multimodal learning surveys supply the vocabulary [23]. Evidential fusion is attractive for edge use because its combination rule is closed-form and cheap.

D. From Fused Confidence to Allocation

The fused confidence vector becomes the input to the gate (Section XIV-B). Decisions include which modality to run, whether to run a second one for verification, and whether to ask a human. Policy steering that chooses between acting, asking and requesting intervention [46] and conformal help-seeking [45], [84] show how calibrated uncertainty can drive such discrete choices with guarantees.

XIII. ON-DEVICE ADAPTATION: LoRA, REPLAY, AND CONTINUAL LEARNING

A. Parameter-Efficient Adaptation

Low-rank adaptation. LoRA freezes the base model and learns low-rank updates, cutting trainable parameters by 10,000 times and GPU memory by three times relative to full GPT-3 fine-tuning, with no added inference latency [145]. QLoRA fine-tuned a 65B model on one 48 GB GPU by quantizing the base to 4 bits [146]. For robot policies, LoRA gave the best post-adaptation performance with 1% of full fine-tuning parameters in continual imitation [147]. OpenVLA is designed for LoRA fine-tuning on consumer GPUs [13].

Adapter modules. Adapters insert small bottleneck layers, reaching within 0.4% of full fine-tuning with 3.6% added parameters per task [148]. Prefix-tuning trains only a continuous prompt [149], and (IA)³ showed that parameter-efficient fine-tuning beats in-context learning at lower cost [150]. CLARE routes between task-specific adapters inside a VLA and expands them for new tasks without replay [151]. It is an adapter-level analog of modality routing.

B. Continual Learning and Forgetting

Surveys organize continual learning into replay, regularization and parameter-isolation methods [22]. LIBERO benchmarks lifelong robot learning, and found that sequential fine-tuning can beat dedicated lifelong methods in forward transfer [35]. LOTUS grows a skill library by unsupervised skill discovery, outperforming baselines by over 11% [152]. Progressive networks avoid forgetting by adding capacity per task, but grow without bound [153].

Replay and prioritization. Prioritized replay outperformed uniform replay on 41 of 49 Atari games [154]. CLEAR showed that replay substantially reduces forgetting in continual RL, and that a buffer with random discarding performs nearly as well as an unbounded one [155]; this is good news for memory-limited robots. Exemplar selection [156] and gradient-constrained episodic memory [157] are alternatives. For the orchestrator, the buffer stores routing decisions, confidences and outcomes.

Elastic weight consolidation. EWC penalizes changes to parameters important for earlier tasks, using a diagonal Fisher estimate [158]. It doubles parameter memory but adds little compute. How EWC interacts with LoRA updates is not established and should be tested.

C. Feasibility of On-Device Training

Memory and compute budgets. Training on microcontroller-class devices within 256 KB of SRAM has been demonstrated for small vision models [159]. TinyTL showed that activation memory, not parameter count, is the bottleneck, saving up to 6.5 times memory [160]. MCUNet co-designs models and inference engines for microcontrollers [161]. PockEngine reported up to 15 times speed-up over TensorFlow on a Raspberry Pi and fine-tuned LLaMA-2-7B on a Jetson Orin at 550 tokens per second [162]. Zeroth-order fine-tuning needs only inference-level memory [163], and server-assisted side tuning offloads backpropagation [164], which would violate the no-cloud constraint unless the server is local. Together these suggest that LoRA updates of a sub-billion-parameter orchestrator on a Pi 5 are plausible but slow.

Asynchronous background learning. Because inference and learning compete for the same CPU, updates should run in the background between episodes or on the local computer, with the robot swapping in new adapters when validated. Asynchronous inference in LeRobot kept 73.3% success, against 78.3% for synchronous inference, while completing 19 rather than 9 cubes in 60 seconds [15]. This illustrates the latency–quality trade-off that background learning must also manage.

XIV. ADAPTIVE COMPUTATION, MIXTURE OF EXPERTS, AND MODALITY ROUTING

This section develops the central idea of the framework: treating the orchestrator as a learned, uncertainty-aware, compute-budgeted gate over heterogeneous learners.

A. Mixture of Experts as a Framework for Modality Routing

The mixture of experts originates with Jacobs et al., in which a gating network assigns inputs to specialist networks trained jointly with it [165]. Sparsely gated MoE scaled this to more than a thousandfold capacity increases with little loss of efficiency [166]. Switch Transformers simplified routing to a single expert per token with load balancing [167], V-MoE matched state-of-the-art vision models with as little as half the inference compute [168], and Soft MoE replaced hard routing with differentiable slot mixing [169]. Routing networks trained the router with RL to select modules per input, reaching

cross-stitch accuracy with 85% less training time [170]. In robotics, MoE appears inside policies. Sparse Diffusion Policy activates task-specific experts and adds experts for new tasks [171]. MoDE routes by diffusion noise level with 90% lower inference cost [172]. MoE-LoCo runs mixture-of-experts locomotion onboard at 50 Hz on a Jetson Orin [38], and SMP uses sticky routing to activate a small expert subset on a dual-arm robot [173]. Adaptive computation inside VLAs includes early exit under compute and memory budgets, which cut LLM computation by 5.2–6.5 times [174], and layer skipping by a router, which cut compute by up to 5.6 times [175].

These works share a limitation that defines the orchestration gap: the experts are sub-networks of one model, trained jointly and routed on internal features. In the framework considered here, the experts are separate learners of different kinds (perception, RL, imitation, fault adaptation) that expose calibrated confidences, have very different costs, and continue to learn.

B. The Gate Function

Gate input: fused confidence signals. The gate's input is the context (task, observation embedding) together with the calibrated confidence vector $\mathbf{c} = (c_1, \dots, c_4)$ from Section XII and the remaining compute budget. Using confidences as inputs follows LLM routing, where routers predict which of two models will answer well. RouteLLM learns such routers from preference data with cost reductions of over two times [36]. Hybrid LLM routed up to 40% of queries away from the large model with no quality drop, using a router that ran in about 0.036 s [89]. AutoMix uses the small model's self-verification as a confidence signal [90], and FrugalGPT's cascade matched GPT-4 with up to 98% cost reduction [176].

Gate output: routing weights or hard selection. On a robot, the gate usually makes a hard selection, because two policies cannot command the actuators at once. Soft weights remain useful for fusing perception outputs or deciding whether to run a second modality as a verifier. ThriftyDagger's gating between robot and human, based on novelty and risk under an intervention budget, is a two-expert instance of hard selection [42].

Compute budget and an illustrative simulation. The budget can be imposed as a penalty λ on cost or as a hard constraint over a time window (Section XIV-F). Fig. 6(a) shows an illustrative simulation, written for this review, of a four-modality router. Each of four situation types has one modality that succeeds 85% of the time, against 35% for the others, and the modalities cost 1, 3, 2 and 4 units per call. Routing on calibrated confidence minus λ times cost reached 0.85 success at a mean cost of 2.5, equal to an oracle that knows the true success probabilities. A fixed single modality and random routing reached about 0.48. Invoking all four modalities reached 0.96, but only as an idealized upper bound, because a robot cannot execute four policies at once, and it cost 10 units. When the RL and imitation confidences were overconfident by 0.35 and 0.30, success fell to 0.81. A per-modality linear recalibration fitted on 4,000 separate calibration steps restored 0.85. Sweeping λ traces

the success–cost trade-off, and miscalibration shifts the whole curve down. Fig. 6(b) adds the meta-learning experiment of Section XI-E. The simulation is synthetic and makes no claim about robot performance; it shows that calibration errors translate directly into routing errors and that a meta-learned calibration prior shortens the period in which they occur (full results in Appendix B; code in the supplementary material).

C. Conditional Execution and Cascading

Adaptive computation time learns how many steps to spend per input [177], and conditional computation frames unit activation as an RL policy with a cost penalty [178]. Cascades run a cheap model first and escalate only when needed [90], [176]. Fast–slow anomaly handling does the same for failures [88], and resource-aware reasoning learns when to invoke an LLM at all [87]. For edge orchestration, cascading suggests a default policy: run cheap perception and a fast imitation policy, and escalate to RL refinement or fault recovery only when their calibrated confidences fall below a threshold.

D. Load Balancing and Expert Utilization

MoE training suffers from collapse onto a few experts, which auxiliary load-balancing losses counteract [166], [167]. In modality routing, balance is not a goal in itself, because some modalities should be rare. But collapse is still a risk: a modality that is never invoked never generates data, so its confidence is never updated. Exploration bonuses for rarely used modalities, or optimism in the face of uncertainty, address this, following bootstrapped exploration [63].

E. The Orchestration Gap

Table IV and Fig. 4 locate the gap. Uncertainty-aware planners [45], [46] are calibrated but choose between acting and asking, not between learners. Routers [36], [89] are learned and budget-aware but route between LLMs. Adaptive-compute VLAs [174], [175] and adapter routers [121], [151] route inside one model. The unoccupied position is a small orchestrator that is learned online on the device, uses calibrated confidences, respects a compute budget and routes between heterogeneous learning modalities. The novelty lies in this composition, not in any single component, so any system that claims it must show that the composition performs better than its strongest parts used alone.

F. Formal Problem Statement

At each decision step t , the agent observes context x_t (task, observation embedding, robot health) and calibrated confidences $c_t = (c_t^1, \dots, c_t^K)$ from K modalities, where c_t^k estimates the probability that invoking modality k now succeeds. The orchestrator π_θ selects a set $S_t \subseteq \{1, \dots, K\}$, typically a single modality, incurring cost $C(S_t) = \sum_{k \in S_t} \kappa_k$ and receiving task reward r_t . The objective is to maximize expected return subject to an average budget B :

$$\max_{\theta} \mathbb{E} \left[\sum_t \gamma^t r_t \right] \quad \text{s.t.} \quad \mathbb{E} \left[\frac{1}{T} \sum_t C(S_t) \right] \leq B. \quad (2)$$

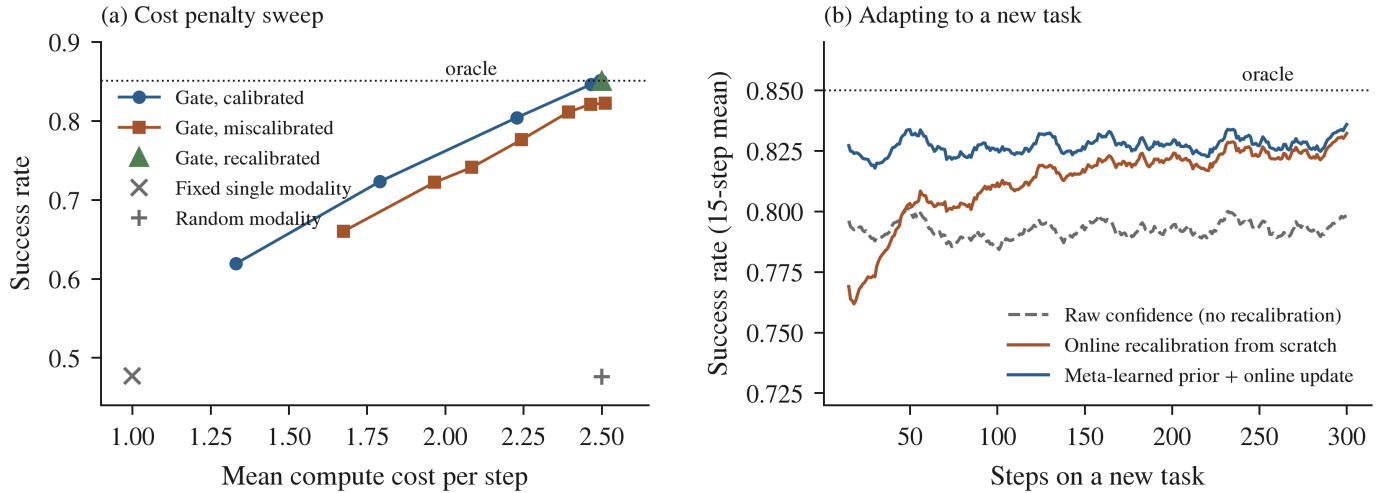


Fig. 6. Illustrative simulation (synthetic data, not robot data). (a) Success rate against mean compute cost for confidence-gated routing among four modalities as the cost penalty λ varies, with calibrated and miscalibrated confidences, recalibration and two baselines; ten seeds of 20,000 steps, standard deviations below 0.005. (b) Success on a new task when the gate uses raw confidences, learns each modality’s calibration offset from scratch, or starts from a prior meta-learned on 50 training tasks; mean of 900 test tasks (three seeds of 300), 15-step moving average.

This is a constrained Markov decision process. Its Lagrangian relaxation, maximizing $\mathbb{E}[\sum_t \gamma^t(r_t - \lambda C(S_t))]$ with $\lambda \geq 0$ updated by dual ascent, gives the cost-penalized gate of Section XIV-B. When a routing choice has no effect beyond its step, the problem reduces to a budgeted contextual bandit in which the confidences act as a learned prior over each arm’s success. Three results frame what can be proved. First, if the confidences are calibrated, greedy selection of $\arg \max_k (c_t^k - \lambda \kappa_k)$ is optimal for the one-step penalized objective. Second, if every confidence is within ε of the true success probability, the greedy choice loses at most 2ε of expected success per step relative to that optimum (Appendix A-B). Third, calibration and exploration must be maintained for modalities that are rarely chosen (Section XIV-D). Meta-learning enters through θ and the calibration maps: both are trained across a task distribution by (1) and adapted online from replayed tuples (x_t, c_t, S_t, r_t) .

XV. LOW-COST PLATFORMS, BENCHMARKS, AND EVALUATION

A. A Reference Low-Cost Platform

Hardware and compute constraints. LeKiwi is an open-source, low-cost mobile manipulator from the LeRobot ecosystem [15], [16]. According to its public documentation, it combines a three-wheel holonomic base with 4-inch omni wheels driven by Feetech servos and an SO-101 arm (SO-100 in earlier versions). A Raspberry Pi 5 is mounted onboard, and power comes from a 12 V battery or a 5 V power-bank variant. In the standard setup, the Pi streams observations to a laptop over ZeroMQ, where policies run. The compute constraint is therefore set by which components run on the Pi and which run on the local laptop.

Sensors. The documented configuration has two USB RGB cameras, one viewing the workspace and one on the wrist, plus joint-position feedback from the servos. There is no depth

TABLE VII
DOCUMENTED CONFIGURATION OF THE LEKIWI REFERENCE PLATFORM

Component	Documented configuration
Onboard computer	Raspberry Pi 5 (4 GB in the reference build)
Off-board computer	Laptop receiving observations over ZeroMQ
Base	Three-wheel holonomic drive, 4-in omni wheels, Feetech servos
Arm	SO-101 (SO-100 in earlier builds), with a leader arm for teleoperation
Cameras	Two USB RGB cameras (workspace and wrist)
Power	12-V battery or 5-V power-bank variant
Software	LeRobot [15], [16]

sensor, which excludes voxel-based policies such as PerAct without additional hardware [96].

B. Benchmark Tasks and Task Distributions

Established benchmarks provide reference points but not the modality-switching structure that orchestration research needs. LIBERO [35], CALVIN [135], RL Bench [136], Meta-World [134] and robomimic [117] are simulated or fixed-arm. The task distribution should therefore be designed so that each modality is the best choice in some situations: novel objects (perception), refinable skills (RL), newly demonstrated behaviors (imitation) and induced faults such as wheel slip, payload change or camera occlusion (fault adaptation).

C. Evaluation Protocol

Evaluation best practice for real robots recommends pre-registered protocols, blinded or randomized trial order, reporting of all trials and confidence intervals [179]. Sequential testing can reduce the number of real-robot trials needed to compare two policies by up to 32% [180]. The large-behavior-model study shows how blinded, randomized evaluation is run at scale [181]. With about 50 trials, a success rate of 0.7 has a 95% confidence interval of roughly ± 0.13 , so

differences between conditions smaller than about 0.2 will not be detectable. The trial plan should be sized with this in mind.

Following recommended practice for real-robot evaluation [179], [180], each experiment should specify:

- Tasks and success criteria, defined before data collection.
- Conditions (orchestrator variants and baselines) and their order, randomized across trials.
- Fault-injection procedure and timing.
- Number of trials per condition, chosen by power analysis or sequential testing.
- Reset procedure and environment variation.
- The logging needed to reproduce routing decisions (confidences, costs, selected modality, outcome).

Recommended ablations. Each component of an orchestration system should be ablated: removing calibration so that the gate uses raw signals; using a single confidence signal instead of four; freezing the orchestrator with no LoRA updates; removing replay or the forgetting penalty; removing the compute budget ($\lambda = 0$); replacing the learned gate with hard-coded rules; and removing each modality in turn. Failure cases should be categorized by cause: wrong modality chosen, correct modality failed, calibration drift, or budget exhausted.

D. Comparable Low-Cost Mobile Manipulation Systems

Mobile ALOHA [120] and Hello Robot’s Stretch [182] are the closest low-cost mobile manipulators, both with laptop- or PC-class onboard compute. ACT [6] and SmolVLA [103] are the policies native to the LeRobot stack, and Open X-Embodiment supplies cross-embodiment pretraining data [98]. None of these systems orchestrates learning modalities.

XVI. ANSWERS TO THE RESEARCH QUESTIONS

RQ1: modalities and their signals. Four modalities cover the situations a long-lived robot meets: perception, RL, imitation and fault adaptation (Section IV). Each can emit a confidence signal at low cost: epistemic uncertainty from evidential outputs or small ensembles (Section VII-B); policy entropy or critic disagreement (Section VIII-B); action spread or density scores for imitation (Section IX-B); and an empirical recovery rate for fault adaptation (Section X-C). The signals exist in the literature, but 106 of the 166 non-review works expose none, and policy variance is used as a confidence signal by only one (Fig. 4).

RQ2: how the choice is made today. Choice among skills, models or experts is made by frozen LLM planners [9], [68], by conformal help-seeking that chooses between acting and asking [45], [46], by learned routers between LLMs [36], [89], [90] and by gates inside one network [38], [172], [175]. Table IV shows that no approach satisfies all four requirements of learned, calibrated, budgeted and edge-deployable orchestration, and none arbitrates between more than two heterogeneous learning modalities.

RQ3: signals cheap and calibrated enough to compare. Temperature-scaled or evidential perception, entropy or small critic ensembles for RL, density or reconstruction scores for imitation and Beta-posterior recovery rates are all cheap enough for a Pi-class robot (Table III). None is calibrated

against a shared success variable across modalities, and the simulation shows that this gap costs routing success directly (0.85 against 0.81). Cross-modality calibration is therefore the first problem to solve (Section XVII-A).

RQ4: meta-learning and on-device adaptation. The pieces exist: inner updates that need no gradients (RL², posterior inference, prototypes) or only first-order gradients (Reptile), adapters and LoRA updates that fit in small memory [121], [145], memory-light on-device training [159], [160], [162] and replay with random eviction [155]. Real-robot evidence for meta-learned adaptation at Pi-class cost is limited to a single learner [41], and on-device adaptation is evaluated almost only in simulation (19 of 23 works). Meta-learning a calibration prior is a cheap first step (Section XI-E).

RQ5: distance from the target, and how to close it. The evidence stops short of a resource-constrained robot: of 65 real-robot studies, five run on a Pi-class processor and two of these expose a confidence signal; no work reaches deployment (E6); and only 36 of 161 primary studies report compute as a number (Table II). A claim of learned orchestration should be tested by the protocol of Section XV-C, with ablations that remove calibration, meta-learning, the budget and each modality in turn, and should show that the orchestrated system outperforms its strongest single modality at equal compute.

XVII. OPEN CHALLENGES

A. Online Calibration Across Modalities

The central open problem is placing four heterogeneous confidence signals on one calibrated scale, and keeping it calibrated as the modalities learn and the robot changes. Conformal prediction gives distribution-free guarantees [45], [138], but its exchangeability assumption is broken by an orchestrator that changes which data each modality sees. Online recalibration with drift detection and evidential fusion [143] are promising directions.

B. Training While Acting

LoRA [145] and QLoRA [146] were designed for offline fine-tuning. Updating an orchestrator while it controls a robot raises questions of when to update, how to validate an adapter before deployment, how to avoid forgetting [155], [158], and how to share the CPU with inference [15], [162].

C. Safety of Self-Modifying Learners

An orchestrator that retrains itself can learn unsafe routing. Safe learning in robotics [27], [28] offers several protections. Shields synthesized from temporal logic override unsafe actions [34]. The neural simplex architecture switches to a verified controller while the learner retrains [183]. Safety filters provide a unified framework [184], [185]. A practical design keeps a fixed, verified fallback (for example stop and ask) that the orchestrator cannot unlearn, and validates new adapters before swapping them in.

D. Generalization Beyond the Task Distribution

Meta-learned orchestrators generalize only within their training distribution [134]. Novelty detection [58], [64] and help-seeking [45], [46] are the safeguards when the robot leaves that distribution.

E. Human-Model Uncertainty as an Orchestration Signal

Many of the orchestrator’s decisions involve a person: whether to ask for help, which question to ask, whether to hand over control and how far the person’s answer can be trusted [42], [45], [46]. The four signals of the framework describe the robot’s learners, not the person. Human answers depend on the order and framing of questions, and trust changes with experience in ways that classical probability models capture poorly; quantum-like models of judgement represent these effects explicitly and are reviewed in the companion survey [19]. The companion paper [17] develops an order-aware human model with classical baselines and tests, in simulation, whether it predicts held-out answers and improves the robot’s choice of questions; its predictive uncertainty is the fifth, dashed input of Fig. 1. Whether such a signal improves orchestration with real people is untested.

F. Integration With Higher-Level Planning

The orchestrator sits below task planning. Integrating it with LLM planners [9], [68], so that the planner’s subgoals condition modality selection and routing failures inform replanning, is a natural extension. Resource-aware reasoning shows that the decision of when to plan can itself be learned [87].

XVIII. CONCLUSION

This systematic review of 180 works asked whether the literature supports a meta-learned orchestrator that lets a resource-constrained robot choose among perception, reinforcement learning, imitation and fault adaptation on calibrated confidence and within a compute budget. It does so only in parts. Calibrated and epistemic confidence signals, conformal help-seeking, budget-aware routers, gates inside policies, meta-learned adaptation and memory-light on-device training all exist. Their combination does not: no included work arbitrates between more than two heterogeneous learning modalities with a learned, calibrated and budgeted gate, and the real-robot evidence rarely reaches edge processors (five of 65 studies) or exposes a confidence signal.

The review frames orchestration as a constrained Markov decision process solved by a meta-learner (Sections XI-C and XIV-F). This framing unifies mixture of experts, cascades and meta-reinforcement learning, and it identifies the parts that must run on the robot: cheap confidence signals, calibration maps and an inner update that needs few or no gradients. The illustrative simulation shows that routing inherits every calibration error and that a calibration prior meta-learned across tasks shortens the period in which those errors occur.

The final conclusion is therefore conditional. Meta-learning orchestration on a resource-constrained robot is supported in principle by the components reviewed and is plausible with

small models and careful calibration, but it has not been demonstrated. The decisive test is a real-robot study, run under the protocol of Section XV-C, showing that the orchestrated system outperforms its strongest single modality at equal compute. The main risks lie in cross-modality calibration drift and in the safety of a learner that modifies itself.

DATA AVAILABILITY

The extraction table, quality appraisal, study summaries, codebook, hand-coding files and search strategy are available at [\[FILL: Zenodo DOI\]](#). The simulation code is provided as supplementary material.

ACKNOWLEDGMENT

[FILL: acknowledgments and funding, or delete this section]

APPENDIX A MATHEMATICAL DETAILS

A. Budgeted Routing as a Constrained MDP

With state $s_t = (x_t, c_t, b_t)$, where b_t is the remaining budget, action S_t , reward r_t and cost $C(S_t)$, the Lagrangian $L(\theta, \lambda) = E[\sum_t \gamma^t (r_t - \lambda C(S_t))] + \lambda B$ is maximized over θ and minimized over $\lambda \geq 0$. A practical algorithm alternates policy-gradient updates of θ (for example PPO [4]) with the dual update $\lambda \leftarrow \max(0, \lambda + \eta (\hat{C} - B))$.

B. One-Step Optimality Under Calibration

If $c^k = P(\text{success} \mid k, x)$ exactly, the expected penalized reward of choosing k is $c^k - \lambda \text{cost}_k$, so $\text{argmax}_k (c^k - \lambda \text{cost}_k)$ is optimal for the one-step objective. If each reported confidence differs from the truth by at most ε , the chosen arm’s true value is within 2ε of the optimum. This bound motivates the calibration step.

C. Calibration Maps

Temperature scaling divides logits by T fitted by maximum likelihood on held-out data [57]. Platt scaling fits $\sigma(a \cdot s + b)$ to a raw score s [113]. The simulation fits a linear map from reported confidence to observed success per modality. Split conformal prediction chooses the threshold $\hat{q}$ as the $[(n+1)(1-\alpha)]/n$ quantile of calibration scores, which gives coverage of at least $1-\alpha$ under exchangeability [45].

D. Recovery-Success Posterior

With a $\text{Beta}(\alpha_0, \beta_0)$ prior and s successes in n recovery attempts for a fault class, the posterior mean $(\alpha_0 + s)/(\alpha_0 + \beta_0 + n)$ is the fault modality’s confidence, and its variance measures how much that confidence can be trusted.

TABLE VIII
ILLUSTRATIVE ROUTING SIMULATION: SUCCESS RATE AND MEAN COMPUTE COST ($\lambda = 0.05$), MEAN $\pm$ SD OVER 10 SEEDS OF 20 000 STEPS (SYNTHETIC DATA)

Policy	Success rate	Mean cost per step
Oracle (true success prob.)	0.851 ± 0.003	2.5
Invoke all four	0.959 ± 0.001	10.0
Fixed single modality	0.477 ± 0.005	1.0
Random modality	0.476 ± 0.004	2.5
Gate on calibrated confidence	0.851 ± 0.003	2.5
Gate on miscalibrated confidence	0.811 ± 0.004	2.4
Gate on recalibrated confidence	0.850 ± 0.003	2.5

E. A Candidate Orchestrator Update Algorithm

- Observe x_t ; query the calibrated confidences c_t from the four modalities (cheap signals every step, expensive ones on demand).
- Select $S_t = \operatorname{argmax}_k \pi_{-\theta}(k \mid x_t, c_t, b_t)$, subject to the fallback shield (Section XVII-C).
- Execute; record $(x_t, c_t, S_t, r_t, \text{outcome})$ in the replay buffer.
- Periodically, in the background: recalibrate each modality's confidence map on recent outcomes; update the LoRA adapter of $\pi_{-\theta}$ on prioritized replay [154] with a forgetting penalty [158]; validate the new adapter on held-out episodes before swapping it in; update λ by dual ascent.

APPENDIX B

ILLUSTRATIVE ROUTING SIMULATION: FULL RESULTS

Table VIII reports the first part of the illustrative simulation of Section XIV-B and Table IX the second part (Section XI-E). The data are synthetic; no robot experiments were run for this review. The simulation code is provided as supplementary material.

Sweep of the cost penalty λ (success rate, mean cost), calibrated gate: $\lambda = 0$: 0.851, 2.50; $\lambda = 0.02$: 0.851, 2.50; $\lambda = 0.05$: 0.851, 2.50; $\lambda = 0.1$: 0.846, 2.47; $\lambda = 0.15$: 0.804, 2.23; $\lambda = 0.2$: 0.723, 1.79; $\lambda = 0.3$: 0.619, 1.33. Miscalibrated gate: $\lambda = 0$: 0.822, 2.51; $\lambda = 0.02$: 0.821, 2.46; $\lambda = 0.05$: 0.811, 2.39; $\lambda = 0.1$: 0.776, 2.24; $\lambda = 0.15$: 0.741, 2.08; $\lambda = 0.2$: 0.722, 1.97; $\lambda = 0.3$: 0.660, 1.68.

Meta-learned calibration. In each task, the confidence of modality k is the true success probability plus an offset $b_k \sim \mathcal{N}(\mu_k, 0.10^2)$, with $\mu = (0, 0.30, 0.25, 0.05)$, plus noise of standard deviation 0.08; situation frequencies vary by task. The gate selects $\operatorname{argmax}_k (c^k - \hat{b}_k - 0.05 \kappa_k)$ with 5% random exploration and updates $\hat{b}_k$ as the running mean of $c^k - y$ over its own choices, shrunk toward a prior mean with n_0 pseudo-observations. From scratch, the prior mean is zero and $n_0 = 1$; the meta-learned prior is estimated from 50 training tasks of 400 logged steps each, by empirical Bayes ($\hat{\mu} = (0.01, 0.27, 0.22, 0.03)$, n_0 between 30 and 47). Table IX gives the mean success over 900 new tasks of 300 steps.

TABLE IX
META-LEARNED CALIBRATION: MEAN SUCCESS RATE ON NEW TASKS BY PHASE (SYNTHETIC DATA; 900 TASKS; ORACLE 0.850)

Gate	Steps 1–25	26–100	101–300
Raw confidence	0.793	0.791	0.793
Recalibration from scratch	0.770	0.802	0.821
Meta-learned prior + update	0.823	0.826	0.828

REFERENCES

- [1] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: representing model uncertainty in deep learning," in *Int. Conf. on Machine Learning (ICML 2016)*, 2016.
- [2] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Int. Conf. on Machine Learning (ICML 2018)*, 2018.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.
- [5] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: visuomotor policy learning via action diffusion," in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [6] T. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [7] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.
- [8] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: rapid motor adaptation for legged robots," in *Robotics: Science and Systems (RSS 2021)*, 2021.
- [9] M. Ahn, A. Brohan, N. Brown *et al.*, "Do as i can, not as i say: grounding language in robotic affordances," in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [10] S. Yao, J. Zhao, D. Yu *et al.*, "React: synergizing reasoning and acting in language models," in *Int. Conf. on Learning Representations (ICLR 2023)*, 2023.
- [11] T. Schick, J. Dwivedi-Yu, R. Dessi *et al.*, "Toolformer: language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [12] A. Brohan, N. Brown, J. Carbajal *et al.*, "Rt-2: vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [13] M. Kim, K. Pertsch, S. Karamcheti *et al.*, "Openvla: an open-source vision-language-action model," in *Conference on Robot Learning (CoRL 2024)*, 2024.
- [14] K. Black, N. Brown, D. Driess *et al.*, " π_0 : a vision-language-action flow model for general robot control," in *Robotics: Science and Systems XXI (RSS 2025)*, 2024.
- [15] R. Cadene, S. Alibert, F. Capuano *et al.*, "Lerobot: an open-source library for end-to-end robot learning," in *Int. Conf. on Learning Representations (ICLR 2026)*, 2026.
- [16] R. Cadene, S. Alibert, A. Soare *et al.*, "Lerobot: state-of-the-art machine learning for real-world robotics in pytorch," *GitHub repository, Hugging Face*, 2024. [Online]. Available: <https://github.com/huggingface/lerobot>
- [17] Y. Guru and D. K. S. Chauhan, "Order-aware human models for robot questioning and trust: A quantum-like model with classical baselines, simulation studies and quantum-circuit execution," 2026, companion manuscript, in preparation.
- [18] —, "Quantum technologies for autonomous robotics: A systematic survey of methods, evidence, and deployment constraints," 2026, companion manuscript, prepared for ACM Computing Surveys.
- [19] —, "Quantum-like cognition and decision-making for autonomous agents, from robotics to AI/ML systems: A systematic review," 2026, companion manuscript, prepared for Artificial Intelligence Review.
- [20] J. Gawlikowski, C. Tassi, M. Ali *et al.*, "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, 56(Suppl 1), 1513–1589, 2023.
- [21] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, "Meta-learning in neural networks: a survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5149–5169, 2022.

- [22] M. De Lange, R. Aljundi, M. Masana *et al.*, “A continual learning survey: defying forgetting in classification tasks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.
- [23] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: a survey and taxonomy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.
- [24] P. Kairouz, H. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [25] J. Qi, Q. Zhou, L. Lei, and K. Zheng, “Federated reinforcement learning: techniques, applications, and open challenges,” *Intelligence & Robotics*, vol. 1, no. 1, pp. 18–57, 2021.
- [26] T. Osa, J. Pajarinen, G. Neumann, P. Bagnell, P. Abbeel, and J. Peters, “An algorithmic perspective on imitation learning,” *Foundations and Trends in Robotics*, vol. 7, no. 1–2, pp. 1–179, 2018.
- [27] L. Brunke, M. Greeff, A. Hall *et al.*, “Safe learning in robotics: from learning-based control to safe reinforcement learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.
- [28] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.
- [29] E. Khalastchi and M. Kalech, “On fault detection and diagnosis in robotic systems,” *ACM Computing Surveys*, vol. 51, no. 1, p. 9, 2018.
- [30] G. Pang, C. Shen, L. Cao, and A. van den Hengel, “Deep learning for anomaly detection: a review,” *ACM Computing Surveys*, vol. 54, no. 2, p. 38, 2021.
- [31] Z. Yu, B. Wang, P. Zeng *et al.*, “A survey on efficient vision-language-action models,” 2025.
- [32] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.
- [33] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.
- [34] M. Alshiekh, R. Bloem, R. Ehlers *et al.*, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence (AAAI 2018)*, 2018.
- [35] B. Liu, Y. Zhu, C. Gao *et al.*, “Libero: benchmarking knowledge transfer for lifelong robot learning,” in *NeurIPS 2023 Datasets and Benchmarks*, 2023.
- [36] I. Ong, A. Almahairi, V. Wu *et al.*, “Routellm: learning to route llms with preference data,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [37] C. Agia, R. Sinha, J. Yang *et al.*, “Unpacking failure modes of generative policies: runtime monitoring of consistency and progress,” in *Conference on Robot Learning (CoRL 2024)*, PMLR 270, 2024.
- [38] R. Huang, S. Zhu, Y. Du, and H. Zhao, “Moe-loco: mixture of experts for multitask locomotion,” in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2025)*, 2025, pp. 14 218–14 225.
- [39] D. Kalashnikov, A. Irfan, P. Pastor *et al.*, “Qt-opt: scalable deep reinforcement learning for vision-based robotic manipulation,” in *Conference on Robot Learning (CoRL 2018)*, 2018.
- [40] M. Andrychowicz, F. Wolski, A. Ray *et al.*, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [41] A. Nagabandi, I. Clavera, S. Liu *et al.*, “Learning to adapt in dynamic, real-world environments through meta-reinforcement learning,” in *Int. Conf. on Learning Representations (ICLR 2019)*, 2019.
- [42] R. Hoque, A. Balakrishna, E. Novoseller *et al.*, “Thriftydagger: budget-aware novelty and risk gating for interactive imitation learning,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [43] C. Xu, T. Nguyen, E. Dixon *et al.*, “Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [44] Q. Gu, Y. Ju, S. Sun *et al.*, “Safe: multitask failure detection for vision-language-action models,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.
- [45] A. Ren, A. Dixit, A. Bodrova *et al.*, “Robots that ask for help: uncertainty alignment for large language model planners,” in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [46] J. Yuan, Y. Wu, and A. Bajcsy, “When to act, ask, or learn: uncertainty-aware policy steering,” in *Robotics: Science and Systems (RSS 2026)*, 2026.
- [47] A. Radford, J. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Int. Conf. on Machine Learning (ICML 2021)*, 2021.
- [48] M. Shridhar, L. Manuelli, and D. Fox, “Cliport: what and where pathways for robotic manipulation,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [49] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [50] M. Sensoy, L. Kaplan, and M. Kandemir, “Evidential deep learning to quantify classification uncertainty,” in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018.
- [51] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [52] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Int. Conf. on Machine Learning (ICML 2018)*, 2018.
- [53] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
- [54] S. Ross, G. Gordon, and J. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS 2011)*, 2011.
- [55] P. Florence, C. Lynch, A. Zeng *et al.*, “Implicit behavioral cloning,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [56] J. Bongard, V. Zykov, and H. Lipson, “Resilient machines through continuous self-modeling,” *Science*, vol. 314, no. 5802, pp. 1118–1121, 2006.
- [57] C. Guo, G. Pleiss, Y. Sun, and K. Weinberger, “On calibration of modern neural networks,” in *Int. Conf. on Machine Learning (ICML 2017)*, PMLR 70, 2017, pp. 1321–1330.
- [58] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Int. Conf. on Learning Representations (ICLR 2017)*, 2017.
- [59] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2017)*, 2017.
- [60] X. Yuan, T. Mu, S. Tao *et al.*, “Policy decorator: model-agnostic online refinement for large policy model,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [61] L. Ankile, A. Simeonov, I. Shenfeld *et al.*, “From imitation to refinement – residual rl for precise assembly,” in *IEEE ICRA 2025*, 2025.
- [62] A. Wagenmaker, M. Nakamoto, Y. Zhang *et al.*, “Steering your diffusion policy with latent space reinforcement learning,” 2025.
- [63] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy, “Deep exploration via bootstrapped dqn,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [64] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” in *Int. Conf. on Learning Representations (ICLR 2019)*, 2019.
- [65] R. Römer, A. Kobras, L. Worbis, and A. Schoellig, “Failure prediction at runtime for generative robot policies,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.
- [66] B. Wu, B. Lee, K. Daniilidis, B. Bucher, and N. Matni, “Uncertainty-aware deployment of pre-trained language-conditioned imitation learning policies,” 2024.
- [67] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: extracting actionable knowledge for embodied agents,” in *Int. Conf. on Machine Learning (ICML 2022)*, 2022.
- [68] W. Huang, F. Xia, T. Xiao *et al.*, “Inner monologue: embodied reasoning through planning with language models,” in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [69] J. Liang, W. Huang, F. Xia *et al.*, “Code as policies: language model programs for embodied control,” in *IEEE Int. Conf. on Robotics and Automation (ICRA 2023)*, 2023.
- [70] I. Singh, V. Blukis, A. Mousavian *et al.*, “Progprompt: generating situated robot task plans using large language models,” in *IEEE Int. Conf. on Robotics and Automation (ICRA 2023)*, 2023.
- [71] D. Shah, B. Osiniski, B. Ichter, and S. Levine, “Lm-nav: robotic navigation with large pre-trained models of language, vision, and action,” in *Conference on Robot Learning (CoRL 2022)*, 2022.

- [72] C. Song, J. Wu, C. Washington, B. Sadler, W.-L. Chao, and Y. Su, "Llm-planner: few-shot grounded planning for embodied agents with large language models," in *IEEE/CVF Int. Conf. on Computer Vision (ICCV 2023)*, 2023.
- [73] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: composable 3d value maps for robotic manipulation with language models," in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [74] W. Yu, N. Gileadi, C. Fu *et al.*, "Language to rewards for robotic skill synthesis," in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [75] M. Dalal, T. Chiruvolu, D. Chaplot, and R. Salakhutdinov, "Plan-seq-learn: language model guided rl for solving long horizon robotics tasks," in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [76] D. Driess, F. Xia, M. Sajjadi *et al.*, "Palm-e: an embodied multimodal language model," in *Int. Conf. on Machine Learning (ICML 2023)*, 2023.
- [77] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: an open-ended embodied agent with large language models," *Transactions on Machine Learning Research (TMLR)*, 2024.
- [78] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [79] S. Yao, D. Yu, J. Zhao *et al.*, "Tree of thoughts: deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [80] B. Chen, C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao, "Fireact: toward language agent fine-tuning," 2023.
- [81] A. Zeng, M. Liu, R. Lu *et al.*, "Agenttuning: enabling generalized agent abilities for llms," in *Findings of ACL 2024*, 2024.
- [82] W. Yao, S. Heinecke, J. Niebles *et al.*, "Retroformer: retrospective large language agents with policy gradient optimization," in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [83] Y. Zhai, H. Bai, Z. Lin *et al.*, "Fine-tuning large vision-language models as decision-making agents via reinforcement learning," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [84] K. Liang, Z. Zhang, and J. Fisac, "Introspective planning: aligning robots' uncertainty with inherent task ambiguity," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [85] J. Wang, G. He, and Y. Kantaros, "Probabilistically correct language-based multi-robot planning using conformal prediction," 2024, later IEEE RA-L.
- [86] A. Ren, J. Clark, A. Dixit *et al.*, "Explore until confident: efficient exploration for embodied question answering," in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [87] J. Liu, P. Zhao, Z. Kong *et al.*, "When should a robot think? resource-aware reasoning via reinforcement learning for embodied robotic decision-making," 2026.
- [88] R. Sinha, A. Elhafi, C. Agia *et al.*, "Real-time anomaly detection and reactive planning with large language models," in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [89] D. Ding, A. Mallick, C. Wang *et al.*, "Hybrid llm: cost-efficient and quality-aware query routing," in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [90] P. Aggarwal, A. Madaan, A. Anand *et al.*, "Automix: automatically mixing language models," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [91] A. Brohan, N. Brown, J. Carbajal *et al.*, "Rt-1: Robotics transformer for real-world control at scale," in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [92] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari *et al.*, "Octo: an open-source generalist robot policy," in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [93] K. Bousmalis, G. Vezzani, D. Rao *et al.*, "Robocat: a self-improving generalist agent for robotic manipulation," *Transactions on Machine Learning Research (TMLR)*, 2024.
- [94] S. Reed, K. Zolna, E. Parisotto *et al.*, "A generalist agent," *Transactions on Machine Learning Research (TMLR)*, 2022.
- [95] Y. Jiang, A. Gupta, Z. Zhang *et al.*, "Vima: general robot manipulation with multimodal prompts," in *Int. Conf. on Machine Learning (ICML 2023)*, 2023.
- [96] M. Shridhar, L. Manuelli, and D. Fox, "Perceiver-actor: a multi-task transformer for robotic manipulation," in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [97] X. Li, M. Liu, H. Zhang *et al.*, "Vision-language foundation models as effective robot imitators," in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [98] O. X.-E. Collaboration, "Open x-embodiment: robotic learning datasets and rt-x models," in *IEEE Int. Conf. on Robotics and Automation (ICRA 2024)*, 2024.
- [99] M. Sobol Mark, T. Gao, G. Sampaio *et al.*, "Policy agnostic rl: offline rl and online rl fine-tuning of any class and backbone," in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [100] Y. Chen, S. Tian, S. Liu *et al.*, "Confrt: a reinforced fine-tuning method for vla models via consistency policy," in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [101] G. Lu, W. Guo, C. Zhang *et al.*, "Vla-rl: towards masterful and general robotic manipulation with scalable reinforcement learning," 2025.
- [102] H. Li, Y. Zuo, J. Yu *et al.*, "Simplevla-rl: scaling vla training via reinforcement learning," in *Int. Conf. on Learning Representations (ICLR 2026)*, 2026.
- [103] M. Shukor, D. Aubakirova, F. Capuano *et al.*, "Smolvla: a vision-language-action model for affordable and efficient robotics," 2025, hugging Face.
- [104] J. Wen, Y. Zhu, J. Li *et al.*, "Tinyvla: towards fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters (2025)*, 2025.
- [105] P. Budzianowski, W. Maa, M. Freed *et al.*, "Edgevla: efficient vision-language-action models," 2025, iROS 2024 MoMa workshop.
- [106] M. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: optimizing speed and success," in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [107] K. Pertsch, K. Stachowicz, B. Ichter *et al.*, "Fast: efficient action tokenization for vision-language-action models," in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [108] H. Wang, C. Xiong, R. Wang, and X. Chen, "Bitvla: 1-bit vision-language-action models for robotics manipulation," 2025.
- [109] J. Zhang, Y. Hsieh, Z. Wan *et al.*, "Quantvla: scale-calibrated post-training quantization for vision-language-action models," in *IEEE/CVF CVPR 2026*, 2026.
- [110] J. Liu, M. Liu, Z. Wang *et al.*, "Robomamba: efficient vision-language-action model for robotic reasoning and manipulation," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [111] Z. Ravichandran, I. Hounie, F. Cladera *et al.*, "Distilling on-device language models for robot planning with minimal human intervention," 2025.
- [112] A. Amini, W. Schwarting, A. Soleimany, and D. Rus, "Deep evidential regression," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [113] T. Zollo and R. Zemel, "Confidence calibration in vision-language-action models," 2025.
- [114] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [115] E. Jang, A. Irpan, M. Khansari *et al.*, "Bc-z: zero-shot task generalization with robotic imitation learning," in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [116] C. Lynch, M. Khansari, T. Xiao *et al.*, "Learning latent plans from play," in *Conference on Robot Learning (CoRL 2019)*, 2019.
- [117] A. Mandlekar, D. Xu, J. Wong *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [118] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [119] Y. Duan, M. Andrychowicz, B. Stadie *et al.*, "One-shot imitation learning," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [120] Z. Fu, T. Zhao, and C. Finn, "Mobile aloha: learning bimanual mobile manipulation with low-cost whole-body teleoperation," in *Conference on Robot Learning (CoRL 2024)*, 2024.
- [121] R. Zhu, E. Sun, G. Huang, and O. Celiktutan, "Efficient continual adaptation of pretrained robotic policy with online meta-learned adapters," 2025.
- [122] Z. Liu, A. Bahety, and S. Song, "Reflect: summarizing robot experiences for failure explanation and correction," in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [123] J. Duan, W. Pumacay, N. Kumar *et al.*, "Aha: a vision-language-model for detecting and reasoning over failures in robotic manipulation," in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [124] Z. Ye, W. Lu, M. Ye, T. Lin, S. Yang, J. Yan, and B. Zhao, "Robofac: a comprehensive framework for robotic failure analysis and correction," 2025.

- [125] L. Shi, Z. Hu, T. Zhao *et al.*, “Yell at your robot: improving on-the-fly from language corrections,” in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [126] H. Liu, S. Nasiriany, L. Zhang *et al.*, “Robot learning on the job: human-in-the-loop autonomy and learning during deployment,” in *Robotics: Science and Systems (RSS 2023); IJRR 2025*, 2023.
- [127] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Int. Conf. on Machine Learning (ICML 2017)*, 2017.
- [128] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms,” 2018.
- [129] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [130] M. Andrychowicz, M. Denil, S. Gomez *et al.*, “Learning to learn by gradient descent by gradient descent,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [131] Y. Duan, J. Schulman, X. Chen, P. Bartlett, I. Sutskever, and P. Abbeel, “ R^2 : fast reinforcement learning via slow reinforcement learning,” 2016.
- [132] K. Rakelly, A. Zhou, D. Quillen, C. Finn, and S. Levine, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *Int. Conf. on Machine Learning (ICML 2019)*, 2019.
- [133] L. Zintgraf, K. Shiarlis, M. Igl *et al.*, “Varibad: a very good method for bayes-adaptive deep rl via meta-learning,” in *Int. Conf. on Learning Representations (ICLR 2020)*, 2020.
- [134] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: a benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on Robot Learning (CoRL 2019)*, 2019.
- [135] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “Calvin: a benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [136] S. James, Z. Ma, D. Arrojo, and A. Davison, “Rlbench: the robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [137] R. Römer, Seeliger, et al. (8 authors, last Schoellig, and A.P.), “Uncertainty quantification for flow-based vision-language-action models,” 2026.
- [138] L. Lindemann, M. Cleaveland, G. Shim, and G. Pappas, “Safe planning in dynamic environments using conformal prediction,” *IEEE Robotics and Automation Letters*, 2023.
- [139] H. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS 2017)*, 2017.
- [140] T. Li, A. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. Machine Learning and Systems (MLSys 2020)*, 2020.
- [141] A. Kendall, Y. Gal, and R. Cipolla, “Multi-task learning using uncertainty to weigh losses for scene geometry and semantics,” in *IEEE/CVF CVPR 2018*, 2018, pp. 7482–7491.
- [142] Z. Han, C. Zhang, H. Fu, and J. Zhou, “Trusted multi-view classification,” in *Int. Conf. on Learning Representations (ICLR 2021)*, 2021.
- [143] —, “Trusted multi-view classification with dynamic evidential fusion,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [144] Z. Han, F. Yang, J. Huang, C. Zhang, and J. Yao, “Multimodal dynamics: dynamical fusion for trustworthy multimodal classification,” in *IEEE/CVF CVPR 2022*, 2022, pp. 20 707–20 717.
- [145] E. Hu, Y. Shen, P. Wallis *et al.*, “Lora: low-rank adaptation of large language models,” in *Int. Conf. on Learning Representations (ICLR 2022)*, 2022.
- [146] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [147] Z. Liu, J. Zhang, K. Asadi *et al.*, “Tail: task-specific adapters for imitation learning with large pretrained models,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [148] N. Houlsby, A. Giurgiu, S. Jastrzebski *et al.*, “Parameter-efficient transfer learning for nlp,” in *Int. Conf. on Machine Learning (ICML 2019)*, 2019.
- [149] X. Li and P. Liang, “Prefix-tuning: optimizing continuous prompts for generation,” in *Proc. ACL-IJCNLP 2021*, 2021.
- [150] H. Liu, D. Tam, M. Muqeeth *et al.*, “Few-shot parameter-efficient finetuning is better and cheaper than in-context learning,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- [151] R. Römer, Y. Zhang, and A. Schoellig, “Clare: continual learning for vision-language-action models via autonomous adapter routing and expansion,” 2026.
- [152] W. Wan, Y. Zhu, R. Shah, and Y. Zhu, “Lotus: continual imitation learning for robot manipulation through unsupervised skill discovery,” in *IEEE ICRA 2024*, 2024.
- [153] A. Rusu, N. Rabinowitz, G. Desjardins *et al.*, “Progressive neural networks,” 2016.
- [154] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Int. Conf. on Learning Representations (ICLR 2016)*, 2016.
- [155] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, “Experience replay for continual learning,” in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.
- [156] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. Lampert, “icarl: incremental classifier and representation learning,” in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR 2017)*, 2017.
- [157] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continual learning,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [158] J. Kirkpatrick, R. Pascanu, N. Rabinowitz *et al.*, “Overcoming catastrophic forgetting in neural networks,” in *Proceedings of the National Academy of Sciences*, 114(13), 2017, pp. 3521–3526.
- [159] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, “On-device training under 256kb memory,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- [160] H. Cai, C. Gan, L. Zhu, and S. Han, “Tinytl: reduce memory, not parameters for efficient on-device learning,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [161] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, “McuNet: tiny deep learning on iot devices,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [162] L. Zhu, L. Hu, J. Lin *et al.*, “Pockengine: sparse and efficient finetuning in a pocket,” in *IEEE/ACM MICRO 2023*, 2023.
- [163] S. Malladi, T. Gao, E. Nichani *et al.*, “Fine-tuning language models with just forward passes,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [164] L. Li, X. Yang, W. Wu, H. Wang, T. Ohtsuki, X. Fu, M. Pan, and X. Shen, “Mobillm: enabling llm fine-tuning on the mobile device via server assisted side tuning,” 2025.
- [165] R. Jacobs, M. Jordan, S. Nowlan, and G. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [166] N. Shazeer, A. Mirhoseini, K. Maziarz *et al.*, “Outrageously large neural networks: the sparsely-gated mixture-of-experts layer,” in *Int. Conf. on Learning Representations (ICLR 2017)*, 2017.
- [167] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [168] C. Riquelme, J. Puigcerver, B. Mustafa *et al.*, “Scaling vision with sparse mixture of experts,” in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.
- [169] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, “From sparse to soft mixtures of experts,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [170] C. Rosenbaum, T. Klinger, and M. Riemer, “Routing networks: adaptive selection of non-linear functions for multi-task learning,” in *Int. Conf. on Learning Representations (ICLR 2018)*, 2018.
- [171] Y. Wang, Y. Zhang, M. Huo, R. Tian, X. Zhang, Y. Xie *et al.*, “Sparse diffusion policy: a sparse, reusable, and flexible policy for robot learning,” in *Proc. 8th Conference on Robot Learning (CoRL 2024)*, PMLR 270, 2024, pp. 649–665.
- [172] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, “Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [173] C. Hao, X. Zhai, Y. Liu, and H. Soh, “Abstracting robot manipulation skills via mixture-of-experts diffusion policies,” 2026.
- [174] Y. Yue, Y. Wang, B. Kang *et al.*, “Deer-vla: dynamic inference of multimodal large language models for efficient robot execution,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [175] R. Zhang, M. Dong, Y. Zhang *et al.*, “Mole-vla: dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation,” in *Proc. AAAI Conference on Artificial Intelligence (AAAI 2026)*, 2026.

- [176] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: how to use large language models while reducing cost and improving performance,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [177] A. Graves, “Adaptive computation time for recurrent neural networks,” 2016.
- [178] E. Bengio, P.-L. Bacon, J. Pineau, and D. Precup, “Conditional computation in neural networks for faster models,” 2015, iCLR 2016 Workshop.
- [179] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy *et al.*, “Robot learning as an empirical science: best practices for policy evaluation,” 2024.
- [180] D. Snyder, A. Hancock, A. Badithela *et al.*, “Is your imitation learning policy better than mine? policy comparison with near-optimal stopping,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [181] TRI LBM Team, J. Barreiros, A. Beaulieu *et al.*, “A careful examination of large behavior models for multitask dexterous manipulation,” *Science Robotics*, 2025.
- [182] C. Kemp, A. Edsinger, H. Clever, and B. Matulevich, “The design of stretch: a compact, lightweight mobile manipulator for indoor human environments,” in *IEEE ICRA 2022*, 2022.
- [183] D. Phan, R. Grosu, N. Jansen *et al.*, “Neural simplex architecture,” in *NASA Formal Methods (NFM 2020)*, 2020.
- [184] K.-C. Hsu, H. Hu, and J. Fisac, “The safety filter: a unified view of safety-critical control in autonomous systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, 2024.
- [185] K. Wabersich, A. Taylor, J. Choi, K. Sreenath, C. Tomlin, A. Ames, and M. Zeilinger, “Data-driven safety filters: Hamilton-jacobi reachability, control barrier functions, and predictive methods for uncertain systems,” *IEEE Control Systems Magazine*, 2023.

Yeshwanth Guru [FILL: biography: education, current position at the Department of Mechanical Engineering, Amrita Vishwa Vidyapeetham, Chennai, and research interests]

Dev Kunwar Singh Chauhan [FILL: biography: education, current position and research interests]